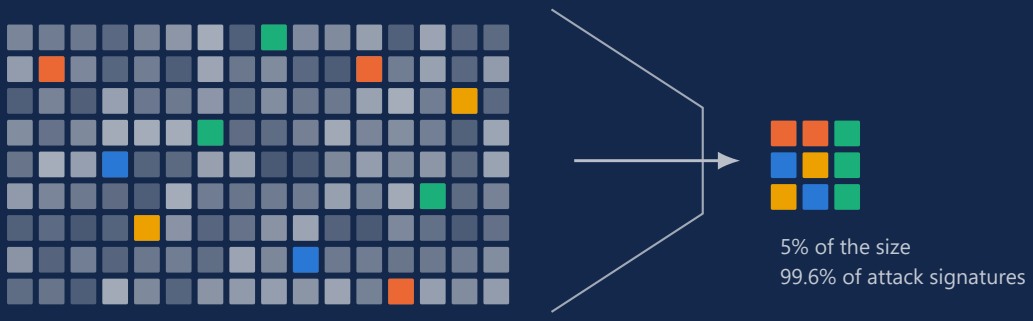
THE COMPLETE GUIDE

Evidence-Based Retention



How to shrink huge recordings of network traffic without losing the evidence, explained from the very first idea to the final result.

Written for complete beginners. No background in networking or security is needed.

Ishaan Suresh Mullya · **Shriya P Reddy** · **Om KB**

Information Security Lab (ICT3141), B.Tech. CCE, 5th Semester

Faculty guide: Dr. Adesh ND · Academic Year 2026–2027

How to read this book

This book assumes you know nothing about networks or security. Every new word is explained the first time it appears, and every word is collected in the glossary at the back.

The project in this book answers one practical question: **if a company cannot afford to keep every recording of its network traffic, which parts should it keep so that it can still investigate an attack later?**

To follow the answer you need a few building blocks, so the book starts from those. Here is the route.

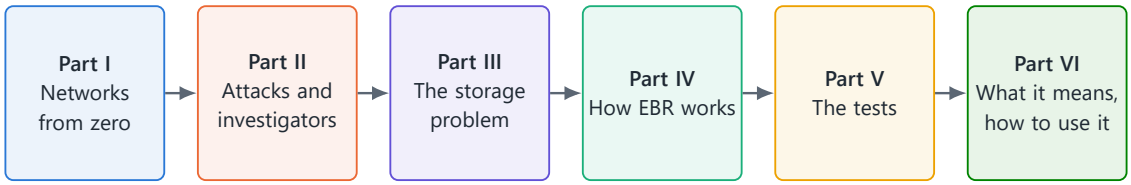


Figure 0.1. The route through this book. If you already know networking, start at Part III.

The coloured boxes

Five kinds of box appear throughout. You can skim them or read them closely.

> **BIG IDEA**
>
> The main point of a section, in a sentence or two.

> **EVERYDAY PICTURE**
>
> A comparison with something from ordinary life. These explain the idea, and they are not exact.

> **NEW WORDS**
>
> Technical terms, defined in plain language.

> **WORKED EXAMPLE**
>
> A small calculation you can follow with a pen. The numbers in these boxes were produced by running the project's real code.

> **HONEST NOTE**
>
> A place where a result was weaker than hoped, an idea failed, or a claim should not be pushed too far.

A promise about the numbers

Every result in this book comes from the project's saved results. Where a test went badly, the book says so. Good research reports its failures, and this project had several. Chapter 28 lists them in one place.

If you are short of time

Table 0.1. Where to start, depending on what you need.

You have	Read
Ten minutes	Chapter 1, which tells the whole story in two pages
An hour	Chapters 1, 10, 11 and 13, then the headline results in Chapter 26
A viva tomorrow	Chapters 15–18, 20, 26, 29 and the viva questions in Chapter 33
A demonstration to give	Chapter 32, including its warning about what the demonstration shows

Contents

How to read this book i

1 The whole project in two pages 1

Part I Networks from zero

2 What is a computer network? 4

3 Packets: how data really travels 6

4 Protocols: the rules of conversation 8

5 Flows: from packets to conversations 11

6 Recording traffic: packet capture 13

Part II Attacks and investigators

7 What attackers do 15

8 Intrusion detection: the guard at the door 17

9 Network forensics: the investigation 19

Part III The storage problem

10 Why recordings cannot simply be kept 22

11 What people do today 24

12 The story of this project 26

Part IV How EBR works

13 The idea in one picture 29

14 Step 1: keep part of a flow 31

15 Step 2: measuring surprise in bits 33

16 Step 3: automation and pivoting 36

17 Step 4: diminishing returns 38

18 Step 5: the knapsack 40

19 Step 6: the flow ledger and the output 42

20 Verify before delete 44

Part V The tests

21 How to test fairly 47

22 The lab and the data 50

23 The first sealed test 52

24 The second sealed test 54

25 The verify-before-delete prototype 56

26 The third sealed test 57

27 Which parts of EBR matter? 61

28 Everything we got wrong 62

Part VI What it means, and how to use it

29 What is new, and what is not 64

30 The limits of EBR 66

31 Could this work in the real world? 67

32 Running the project yourself 68

33 Viva questions 70

A Glossary 73

B Formula sheet 76

C Settings 77
References 78

1 The whole project in two pages

Read this first. It gives the complete story, so that everything later has a place to fit. Do not worry if some words are new; each one is explained in later chapters.

1.1 The problem

Computers on a network talk by sending small pieces of data called *packets*. A company can record every packet that crosses its network. That recording is the best evidence it will ever have if it is attacked, because it shows exactly what happened. The trouble is size. A single busy 1 gigabit-per-second connection can produce 10.8 terabytes in one day. Nobody keeps that for long, so the recording is usually deleted within days, often before anyone knows an attack took place.

1.2 The question

Suppose you can only keep 5% of the recording. **Which 5% would an investigator most regret losing?**

1.3 Our answer: Evidence-Based Retention (EBR)

EBR is an algorithm. It reads a recording and decides, for every conversation in it, how much to keep. It keeps almost everything that is unusual or that shows an attacker's behaviour, and it throws away repetition and bulk. It needs no list of known attacks. It works purely from the traffic itself, using an idea from information theory: *a surprising thing carries more information than an expected thing*.

We then added a second idea called **verify before delete**. The original recording still exists until the moment you delete it, so you can check what your reduced copy has lost and repair the damage first.

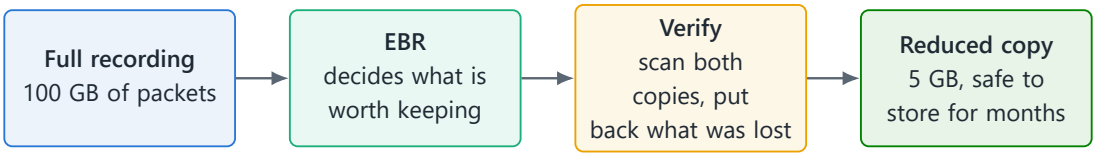


Figure 1.1. The project in one line. The sizes are an illustration of a 5% cap.

1.4 How we tested it

Claims about security tools are easy to make and hard to trust, so the tests were built to be hard to fool:

- **An independent judge.** Suricata, a real intrusion detection system with 52,778 public detection rules, decides what counts as an attack. EBR never sees those rules while choosing what to keep.
- **Real recordings.** We used public captures from research datasets and cyber-defence competitions, 24.5 GB across the three final tests.

- **Sealed tests.** The code and settings were locked with digital fingerprints before the test data was opened, so nothing could be tuned to the answers.
- **Pre-registration.** Before the third test, we wrote down five predictions and the exact rule for passing each one.

1.5 What we found

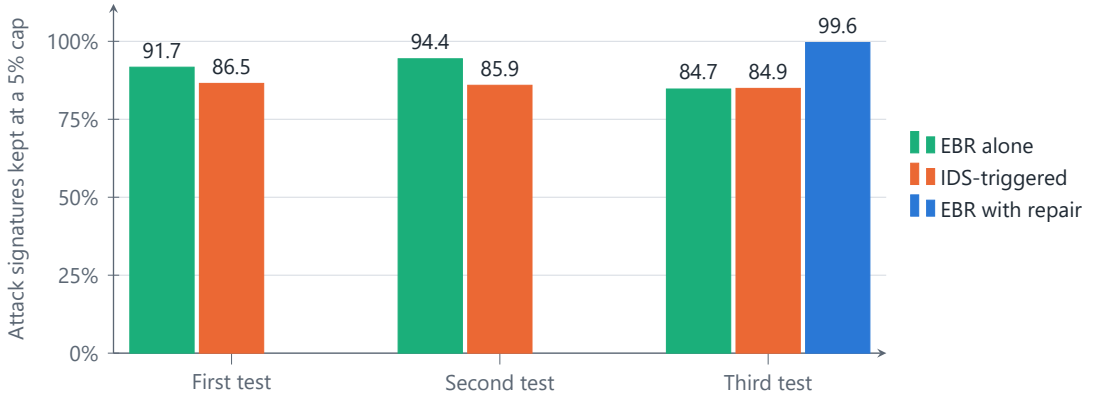


Figure 1.2. Attack signatures still detectable after shrinking to 5% of the original size. "IDS-triggered" is the strong existing method: it uses real detection rules to pick what to keep. Repair was only sealed-tested in the third test.

BIG IDEA · The headline

In a pre-registered sealed test, EBR with verify-before-delete kept **99.6% of attack signatures at a 5% cap**, against 84.9% for the best existing method we compared with, using the same rule knowledge.

HONEST NOTE

Only one of the five predictions passed in full. Three failed and one could not be scored. The test had only 8 scored cases, so this is strong evidence, not proof. Chapter 26 goes through every verdict. Reporting those failures is part of what makes the one clear pass believable.

Part I

Networks from zero

What a network is, how data travels across it, and how that traffic can be recorded.

Computers and addresses · Packets · Protocols · Flows · PCAP files

2 What is a computer network?

A network is a set of computers that can send messages to each other. Almost everything else in this book builds on four ideas: clients, servers, addresses and ports.

2.1 Clients and servers

When you open a website, your laptop asks another computer for a page, and that computer sends it back. The one asking is the **client**. The one answering is the **server**. A server is just a computer whose job is to wait for requests.

Between them sit **routers**, devices that pass messages along towards their destination, much as sorting offices pass along a letter. The internet is a huge number of networks joined by routers.



Figure 2.1. A client asks, a server answers, and routers carry the messages in between.

2.2 IP addresses

Every device needs an address so messages can reach it. This is its **IP address**, usually written as four numbers from 0 to 255, such as `192.168.1.20`. Newer addresses, called IPv6, are longer, but they do the same job.

Some ranges, such as those starting `192.168.` or `10.`, are **private**. They are used inside a home or company and are not reachable directly from the internet. EBR does not need to know this, but one of the methods we compare against does.

2.3 Ports

One computer can run many programs at once: a website, an email service and a file share. A **port** is a number from 0 to 65,535 that says which program a message is for.

> EVERYDAY PICTURE
>
> An IP address is the address of a block of flats. A port is the flat number. The postal service gets the letter to the building, and the flat number gets it to the right person.

Servers use well-known ports so clients know where to knock. Clients usually pick a random high number for their own side, called an **ephemeral port**.

Table 2.1. Well-known ports you will meet in this book.

Port	Service	Readable?	What it does
22	SSH	Encrypted	Secure remote control of a computer
23	Telnet	Plain text	Old remote control. Passwords travel in the clear
25	SMTP	Plain text	Sending email
53	DNS	Plain text	Turning names like `example.com` into IP addresses
80	HTTP	Plain text	Web pages without encryption
443	HTTPS (TLS)	Encrypted	Web pages with encryption
445	SMB	Often readable	Windows file sharing, and a favourite route for attackers moving inside a network

CHAPTER IN SHORT

A client asks and a server answers. Routers carry the messages. An IP address finds the computer, and a port finds the program on it.

3 Packets: how data really travels

Data does not travel as one long stream. It is cut into small numbered pieces. This single fact is why recording traffic is possible, and why so much of it can be thrown away.

3.1 Why data is cut into pieces

A video or a large file is split into small pieces called **packets**, usually no larger than about 1,500 bytes. Each packet finds its own way across the network, and the receiver puts them back in order.

This has real advantages. If one packet is lost, only that piece needs resending. Many people can share one connection, because everyone's packets take turns.

> EVERYDAY PICTURE
>
> Imagine posting a book by sending each page on its own numbered postcard. A card can go missing and only that one needs resending. Your cards can mix with everyone else's in the postbox. At the other end, the numbers let the reader put the pages back in order.

3.2 Inside a packet

A packet has two parts. The **headers** are the addressing and control information, like the writing on an envelope. The **payload** is the actual content, like the letter inside.

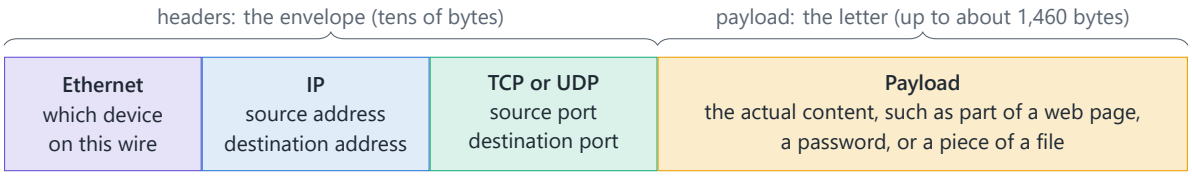


Figure 3.1. A packet is nested like envelopes inside envelopes. Each layer adds its own header.

> NEW WORDS
>
> **Header:** the control information at the front of a packet. **Payload:** the content being carried. **Metadata:** data about data, such as who talked to whom and when, without the content.

3.3 Units of size

Sizes matter throughout this book, so here they are in one place. We use the decimal units that storage makers use.

Table 3.1. Units of data.

Unit	Size	Roughly	Example
Byte	8 bits	One letter	The letter “A”
Kilobyte (KB)	1,000 bytes	A short email	Half a page of text
Megabyte (MB)	1,000 KB	A photo	About 700 full-size packets
Gigabyte (GB)	1,000 MB	An hour of video	About 700,000 full-size packets
Terabyte (TB)	1,000 GB	A large hard drive	A busy link can fill ten of these in a day

Network speeds are measured in *bits* per second, and storage in *bytes*. One byte is 8 bits, which is why a “1 gigabit” link moves 125 megabytes each second, not 1,000.

CHAPTER IN SHORT

Data travels as packets of up to about 1,500 bytes. Each has headers (the envelope) and a payload (the letter). Even when the letter is encrypted, the envelope stays readable.

4 Protocols: the rules of conversation

A protocol is an agreed set of rules for talking. Computers made by different companies can work together only because they follow the same protocols.

4.1 Layers

Networking is organised in layers. Each layer does one job and relies on the layer below it. A web browser does not need to know about cables, and a cable does not need to know about web pages.

Application	What the program is saying. HTTP, DNS, SMB, SSH, Telnet
Transport	Which program, and is delivery reliable? TCP and UDP
Internet	Which computer, anywhere in the world? IP
Link	Which device on this cable or Wi-Fi? Ethernet

Figure 4.1. Four layers of networking. The header sections in the previous chapter's packet diagram match these layers.

4.2 TCP: the reliable conversation

TCP makes sure everything arrives, in order. Before sending data, the two sides shake hands. This three-way handshake is one of the most important patterns in network forensics, because of what it shows when it *fails*.

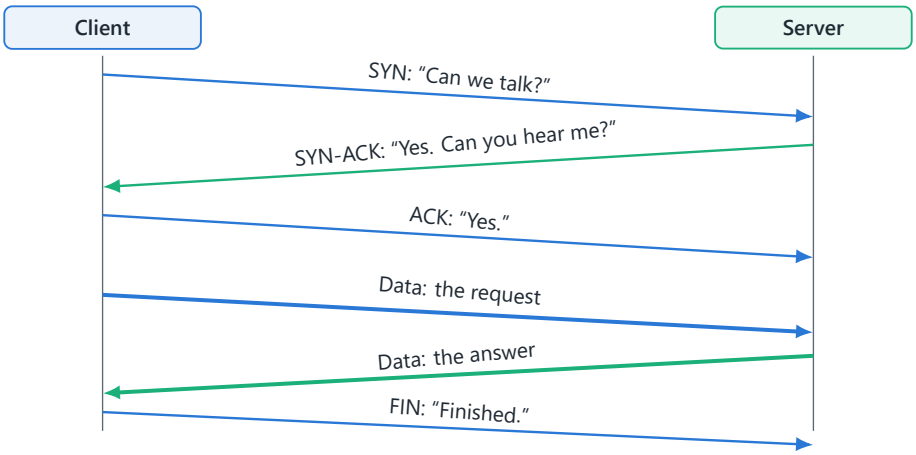


Figure 4.2. A normal TCP conversation. Control messages are carried in packets with little or no payload.

The small labels SYN, ACK, FIN and RST are **flags**, single bits in the TCP header that say what a packet is for. Every byte is also numbered with a **sequence number**, so the receiver can reorder packets and spot gaps. EBR uses these numbers to know where in a conversation each packet belongs, and it does not need to reassemble the whole conversation to do this.

Conversations can end badly, and the way they fail tells a story:

Table 4.1. How EBR labels a TCP conversation.

Label	What happened	What it often means
SF	Normal finish	An ordinary conversation
S0	A SYN was sent and nothing answered	Scanning, or a dead address
REJ	The server refused straight away	Scanning a closed port
RSTO	The connection was reset before any data	An aborted attempt
RST	Reset after some exchange	Either side gave up part way through

4.3 UDP: the quick note

UDP sends a packet and hopes. There is no handshake and no resending. It suits small, quick exchanges such as DNS lookups, and it also suits attackers who want to flood a target.

4.4 The application protocols that matter here

DNS: the phone book

People remember names, and computers need numbers. **DNS** turns `example.com` into an IP address. Malware has to find its controller somehow, so DNS records are valuable evidence.

HTTP: the web, in plain text

A web request is readable text. This is a real request, and every line of it is visible to anyone recording the traffic:

```
GET /page?id=7 HTTP/1.1
Host: shop.example
User-Agent: Mozilla/5.0
```

The first few lines of a conversation often hold the most useful evidence, which is why keeping only the start of a conversation works so well. Chapter 14 builds on this.

TLS: the sealed envelope

TLS encrypts the content, and it is the S in HTTPS. A recording then shows only scrambled bytes. The headers, and some details of how the connection was set up, are still visible.

> **BIG IDEA**
>
> Encrypted traffic is nearly useless to keep in full, because nobody can read it later. Readable traffic is far more valuable per byte. EBR values the two differently.

Telnet, SSH and SMB

Telnet gives remote control of a device in plain text, and many cheap internet-connected devices still use it, often with factory passwords. **SSH** does the same job with encryption. **SMB** shares files on Windows networks and is a common route for attackers moving from one computer to the next.

> **CHAPTER IN SHORT**
>
> Protocols are rules. TCP is reliable and shakes hands first. UDP just sends. DNS finds names, HTTP carries readable web traffic, and TLS seals it. How a conversation starts and ends often tells you more than its middle.

5 Flows: from packets to conversations

Packets are the letters. A flow is the whole exchange. EBR thinks in flows, because investigators do.

5.1 What a flow is

All the packets of one conversation share five values, the **five-tuple**: source address and port, destination address and port, and protocol. Grouping packets by these values gives a **flow**.

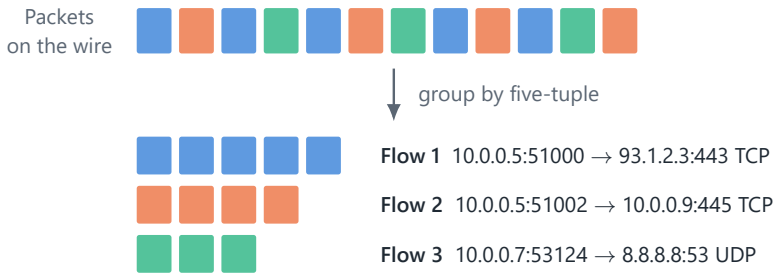


Figure 5.1. Packets from different conversations are mixed together on the wire. The five-tuple sorts them back out.

> EVERYDAY PICTURE
>
> A phone bill lists calls, not words: who called whom, when, and for how long. A flow record is the phone bill of a network. The packets are the recording of the call itself.

A flow ends when it finishes, or after 300 quiet seconds for TCP or 60 for UDP. A fresh handshake also starts a new flow.

5.2 Mice and elephants

> BIG IDEA · Most flows are small, and most bytes are in a few flows
>
> Network people call small flows *mice* and huge ones *elephants*. A movie download is one elephant holding gigabytes. A scan may be a million mice holding almost nothing. Attacks tend to live among the mice and at the start of conversations. Storage is eaten by the elephants.

So dropping the middle of the elephants saves a great deal of space. The hard part is that some mice are noise, and some attacks hide deep inside long sessions. The rest of this book is about getting that decision right.

CHAPTER IN SHORT

A flow is one conversation, identified by its five-tuple. Bytes are concentrated in a few elephants, and evidence is often concentrated in the mice and the opening bytes.

6 Recording traffic: packet capture

Recording every packet is called packet capture. The result is usually a PCAP file, which is the raw material of this entire project.

6.1 How traffic gets recorded

A company places a sensor where traffic passes, such as the connection between its network and the internet. A network switch or a device called a tap copies every packet to the sensor, which writes them to disk. Users do not notice, because the sensor only listens.

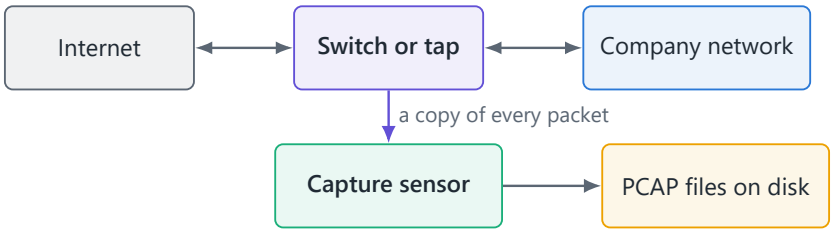


Figure 6.1. The sensor gets a copy of the traffic. Normal traffic is not interrupted.

6.2 The PCAP file

PCAP stands for packet capture. A classic PCAP file is very simple: one 24-byte file header, then each packet in order, with a 16-byte record header giving its time and length.



Figure 6.2. A classic PCAP file. Because the format is this simple, a reduced PCAP is just a shorter list of the same untouched packets.

A newer format called PCAPNG also exists. EBR reads classic PCAP, so PCAPNG files must be converted first, and the project includes a converter. **Wireshark** is the most popular free program for looking inside these files. Open any PCAP in it and you will see the packets, headers, flags and payloads from this part of the book.

> **BIG IDEA**
>
> A reduced PCAP made by EBR is still a normal PCAP. Each kept packet is byte-for-byte identical to the original. Any standard tool, such as Wireshark or Suricata, can open it.

> **CHAPTER IN SHORT**
>
> A sensor copies traffic and writes it to a PCAP file: a header followed by every packet. That file is complete, exact and very large.

Part II

Attacks and investigators

What attackers actually do on a network, how detection software spots them, and how investigators use recordings afterwards.

The stages of an attack · Intrusion detection · Suricata and rules · Network forensics

7 What attackers do

An attack is usually a chain of steps, not one event. Each step leaves different traces in the traffic, and this is why a recording needs to keep many kinds of evidence.

7.1 The chain of an attack

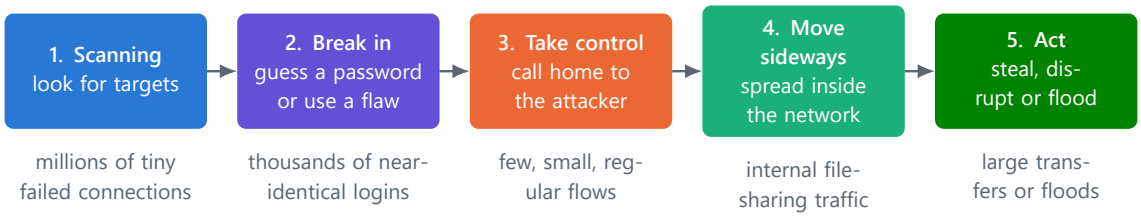


Figure 7.1. A typical attack, with the kind of traffic each step produces.

7.2 The attacks you will meet in this book

Port scanning

A **port scan** tries many ports or addresses to see what answers. It is like walking down a street trying every door handle. Each try is tiny, often a single SYN packet, but there can be millions.

Brute force and login sprays

A **brute force** attack guesses passwords over and over. A **login spray** tries a short list of common passwords against many devices. To a recording, these attempts look almost identical. They have the same size and the same pattern, and the only difference is a few letters in the password. Remember this. It becomes the most important weakness of EBR in Chapter 26.

Exploits and web attacks

An **exploit** takes advantage of a bug. A classic example is **SQL injection**, where text typed into a website is treated as a database command:

```
normal request:   /page?id=7
attack request:   /page?id=1' OR 1=1--
```

Cross-site scripting (XSS) is a related trick that plants code in a page for other visitors to run.

Botnets and command and control

A **botnet** is a group of infected computers controlled by an attacker. Each infected machine, or *bot*, checks in with a **command-and-control (C2)** server for orders. C2 traffic can be very small and very regular, which makes it easy to miss.

Two botnet families appear in our tests. **Mirai** spread through internet-connected devices by trying factory passwords, famously including over Telnet [13]. **Hide and Seek** is a later botnet of the same broad kind.

Denial of service

A **denial-of-service (DoS)** attack overwhelms a service so real users cannot reach it. When many machines do this together it is a **distributed** attack (DDoS). Some floods are huge. Others, such as Slowloris, hold connections open very slowly.

Lateral movement

After getting in, a human attacker moves from computer to computer inside the network, often using SMB. This is called **lateral movement**. It is a sign that a real person is at work, and an investigator wants to see it above almost anything else.

7.3 Red teams

A **red team** is a group of skilled people who attack a network on purpose, to practise. The WRCCDC, a US student cyber-defence competition, publishes recordings of such contests. They are valuable test data because the attackers are real humans who improvise, unlike the scripted attacks of a lab dataset.

> CHAPTER IN SHORT
>
> Attacks come in stages, and each stage looks different in the traffic: floods of tiny failures, repetitive logins, small regular check-ins, internal file sharing. A method that keeps only one kind of traffic will miss the others.

8 Intrusion detection: the guard at the door

An intrusion detection system watches traffic and raises an alert when something looks like an attack. In this project it plays the role of the independent judge.

8.1 What an IDS does

An **intrusion detection system (IDS)** reads packets and compares them with a large list of descriptions of bad behaviour. There are two broad kinds.

Table 8.1. Two ways to detect attacks.

Kind	How it works	Strength	Weakness
Signature	Looks for patterns known to be bad	Precise, and it explains what it found	Blind to anything with no rule yet
Anomaly	Looks for what is unusual	Can notice new things	Many false alarms [12]

8.2 Suricata and its rules

Suricata is a free, widely used signature-based IDS. A **rule** describes what to look for. Here is a simplified rule, written for this book:

```
alert tcp any any -> any 23 (msg:"Example: Telnet login with a default password";
      content:"admin"; sid:1000001; rev:1;)
```

It says: raise an alert for TCP traffic going to port 23 that contains the text `admin`. The `sid` is the rule's ID number.

The tests in this project use **Emerging Threats Open (ET Open)**, a free public rule set. Our copy holds **52,778 rules**. When a rule matches, Suricata writes an **alert** with the rule's name, a category such as "Network Trojan", and the addresses involved.

> NEW WORDS
>
> **Rule (or signature):** a description of bad traffic. **Alert:** what the IDS produces when a rule matches. **False positive:** an alert on harmless traffic. **Attack signature (in this book):** a distinct rule from an attack category that fired on a capture.

8.3 Rate-based rules

Some rules count events. For example, "alert if one address tries 30 ports within 60 seconds". Such a **rate-based rule** needs many flows close together in time. If a reduced capture keeps just a few, spread out, the rule will not fire. This detail shapes two parts of EBR, the exemplar burst in Chapter 17 and the repair neighbours in Chapter 20.

8.4 Rules arrive late

> **BIG IDEA**
>
> Rules are written after someone discovers an attack. A recording made today might contain an attack for which no rule exists until next month. Anything that keeps only what today's rules recognise can never help with that later rule.

This is the main reason EBR refuses to use rules when making its first selection. We tested this idea directly using a "time-travel" measure: alerts from rules published *after* a capture was recorded.

> **CHAPTER IN SHORT**
>
> An IDS compares traffic with thousands of rules. Suricata with ET Open is our judge. It is strong but limited: it only sees what a rule describes, and rules arrive after the attack.

9 Network forensics: the investigation

An IDS raises an alarm in the moment. Forensics is the work that comes afterwards, when someone has to work out what really happened.

9.1 What investigators want to know

Digital forensics means collecting and examining data in a careful way so that its findings can be trusted [14]. In network forensics, the data is traffic. A typical investigation starts with a hint, such as a strange alert or a stolen file, and then works backwards through questions like these:

- **Who?** Which addresses were involved?
- **When?** When did it start, and how long did it last?
- **How?** How did the attacker get in? Which password or flaw did they use?
- **What next?** Which other machines did they touch after that?
- **How much?** What was taken, and how many attempts were made?

> EVERYDAY PICTURE
>
> A packet recording is like CCTV. Most of the time, the footage shows nothing. After a burglary, it is priceless, and only then do you find out which hour you needed. A security office that overwrites its tapes every night will always be short of the footage it wants.

9.2 Pivoting

Investigators do not look at traffic evenly. If one host looks bad, they look at everything it touched. This move is called a **pivot**. EBR copies this habit, as Chapter 16 explains.

9.3 Three kinds of evidence

Network evidence comes in three levels of detail, and each costs a different amount to store.

Table 9.1. Levels of network evidence.

Level	What it holds	What it can answer	Storage
Flow records	Who, when, how much	Who talked to whom, and how many times	Tiny
Packet heads	The first bytes of each conversation	What was asked for, and which software asked	Small
Full packets	Everything	Exactly what was said, in full	Huge

EBR uses all three levels at once. It keeps a compact record of every flow, the heads of most conversations, and full content where that is worth the bytes.

9.4 Hashes: digital fingerprints

A **hash function** turns any file into a short fixed-length string. SHA-256 is the one used here. Change a single bit of the file and the hash changes completely, and nobody can practically find a different file with the same hash.

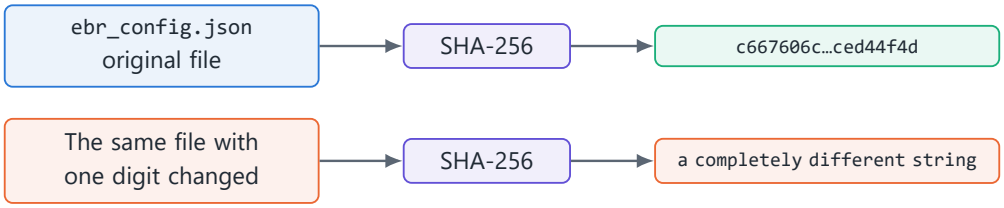


Figure 9.1. A hash is a fingerprint. The top line shows the real hash of our settings file, shortened.

Forensic examiners use hashes to show a file has not changed. We use them for something similar. Before each sealed test we recorded the hashes of the code and settings. Anyone can recompute them to check that nothing was altered afterwards.

> **BE CAREFUL**
>
> A hash proves a file is unchanged. It does not prove the file is true, and it does not by itself make evidence acceptable in court. That needs a documented **chain of custody**, a record of who handled the evidence and when.

> **CHAPTER IN SHORT**
>
> Forensics answers who, when, how and what next. Full recordings answer all of these, but storing them is expensive. Flow records are cheap, and packet heads sit in between. Hashes prove that files have not changed.

Part III

The storage problem

Why a complete recording cannot be kept, what people do about it today, and how this project got to its final idea.

Why compression is not enough · Existing methods · The story of the project

10 Why recordings cannot simply be kept

The arithmetic is short, and it explains the whole field.

10.1 How much data is that?

WORKED EXAMPLE · One day of a 1 gigabit link

A link running flat out at 1 gigabit per second carries:

- 1,000,000,000 bits per second ÷ 8 = 125,000,000 bytes per second, or 125 MB/s
- 125 MB/s × 86,400 seconds in a day = 10,800,000 MB, or **10.8 TB per day**
- 10.8 TB × 30 days = **324 TB per month**

This is a worst case for a link that is always full, and many organisations run several links that are faster than this.

An attack is often noticed long after it starts. If the recording is overwritten every few days, the evidence is already gone by the time anyone looks for it. The storage budget therefore directly limits how far back an investigation can go.

10.2 Why not just compress it?

Lossless compression, such as gzip, shrinks a file by removing repetition, and the original can be rebuilt exactly. It helps, but not nearly enough. Here is what gzip achieved on our own test data.

Table 10.1. What lossless gzip achieved on the three sealed test sets.

Test set	Size	After gzip	Range across captures
First	11.28 GB	51.3%	25.6% to 88.4%
Second	7.03 GB	56.5%	29.1% to 95.6%
Third	6.15 GB	39.9%	14.4% to 73.5%

Encrypted traffic looks random, and random data cannot be compressed. As more traffic becomes encrypted, gzip helps less. One capture barely shrank at all.

BIG IDEA

To reach 1% to 10% of the original size, you have to throw information away on purpose. That is called **lossy reduction**. The whole question becomes: *what can you lose and still investigate?*

10.3 The storage cap

Throughout this project, a method is given a **cap** (or budget): a fixed fraction of the original size. We test four caps: 1%, 2%, 5% and 10%. At a 5% cap, a 1 GB capture must end up at 50 MB or less.

Every byte counts against the cap, including file headers and any summary a method writes. This keeps the comparison fair.

> **CHAPTER IN SHORT**
>
> A busy link produces terabytes a day. Compression alone leaves around 40–55% of the original, far too much. So some data must be deliberately dropped, and the skill lies in choosing which.

11 What people do today

EBR was compared with the strongest simple methods we could build fairly. Each one is shown below as a small picture. Understanding what each loses tells you what EBR has to do better.

11.1 Random packets

Keep each packet with some fixed probability. It is simple and it is hopeless for forensics, because a conversation is left full of holes. Almost nothing can be reassembled. Across our third test it kept only 20.8% of attack signatures at a 5% cap.

11.2 Random flows

Pick whole conversations at random until the budget runs out. What is kept is complete and readable, but the choice is blind. Big flows use up the budget and most small attack flows are lost.

11.3 Flow cutoff (Time Machine)

The **Time Machine** system [5, 6] keeps only the first part of every connection, up to a set number of bytes. This was a major insight: the start of a conversation holds most of its value, and it cuts the elephants down.

We made this comparator stronger than the original by fitting the cutoff exactly to each budget. It still treats every connection the same, and a scan storm can eat the budget. It also loses anything that happens late in a long session.

11.4 Rare destinations (inspired by XNET)

XNET [7] is a recent system that monitors at very high speed. It prioritises rarely contacted destinations and DNS. Our comparator is *inspired by* this idea and is not a copy, since it runs offline and uses no outside popularity lists. It does well when attackers call out to unusual places, and badly when the attack happens inside the network.

11.5 IDS-triggered capture

Here, the IDS decides. Only flows that raised an alert are kept [8]. Commercial and open-source tools offer this, including Suricata's conditional logging. It is very precise, and it is the strongest existing method in this study. We tested a stronger version, **IDS-triggered + fill**, which spends any leftover budget on a flow cutoff.

We also gave it a real advantage: it gets real Suricata alerts from rules published before each capture. EBR gets no rules at all.

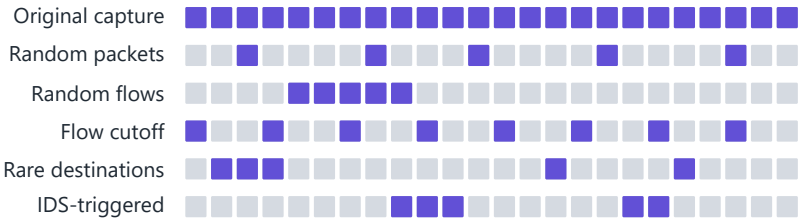


Figure 11.1. Each row is the same capture, and a filled square is a kept packet. These are sketches, not data. Each method spends a similar budget in a very different way.

Table 11.1. The comparators at a glance.

Method	Uses rules?	Strength	Main weakness
Random packets	No	Trivial to run	Destroys conversations
Random flows	No	Kept flows stay whole	Blind choice
Flow cutoff	No	Protects openings of every flow	Treats everything alike
Rare destinations	No	Finds unusual outside contacts	Misses attacks inside the network
IDS-triggered + fill	Yes	Very precise for known threats	Cannot help with rules that do not exist yet

HONEST NOTE

Other work is close to ours, including TrimPCAP [9], which trims the tail of each session, and selective capping [10], which keeps full packets for known protocols and cuts the rest. Keeping data by its value is an old idea too. Chapter 29 sets out carefully what is and is not new here.

CHAPTER IN SHORT

Five comparators cover the main ideas in the field. The strongest is IDS-triggered capture, which uses real rules. Beating it without using rules is hard, and that is the bar we set.

12 The story of this project

The project changed a lot on its way to EBR. Each stage failed in a way that taught something, so it is worth seeing how we got here.

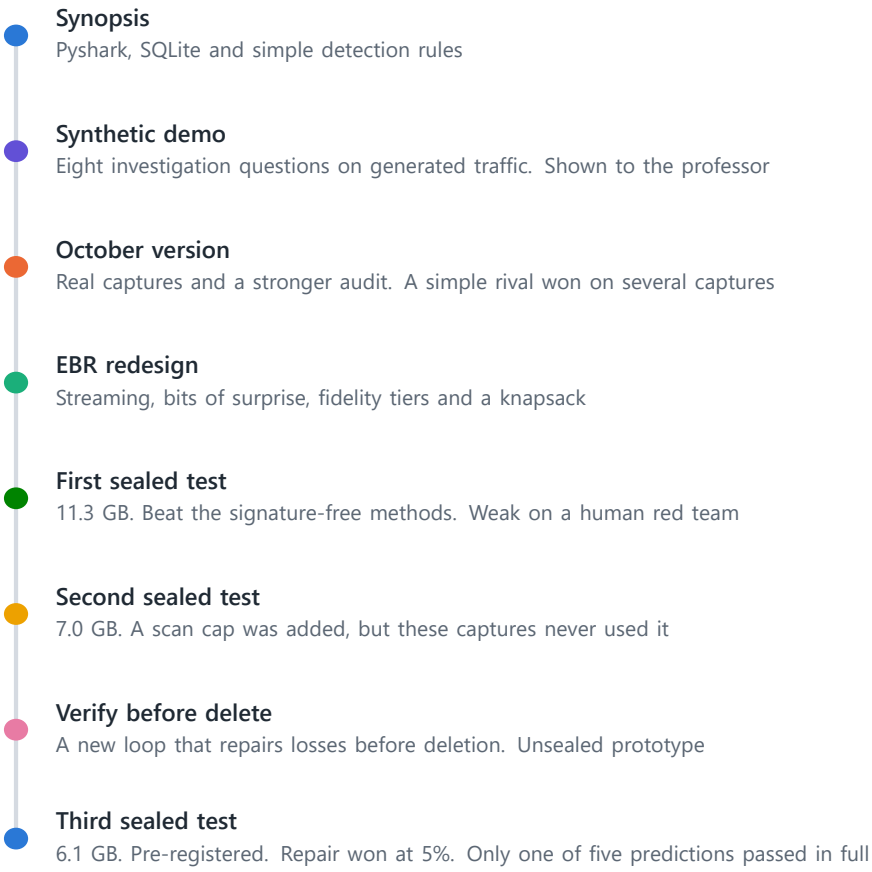


Figure 12.1. How the project developed.

12.1 Stage 1: the synopsis

The project began as a lab mini-project: read PCAP files with a Python library called Pyshark, store the features in a SQLite database, and apply simple rules to spot suspicious behaviour. It worked offline and was meant to teach the basics. The team's first question soon became more interesting than the tool itself: packet files are huge, so which packets matter?

12.2 Stage 2: the synthetic demo

For the progress check we built a demonstration on generated traffic. We wrote eight concrete investigation questions, such as "does the kept capture still show this DNS lookup and its answer?", and scored each method by how many it could still answer. At a 5% cap, the evidence-based method answered 7 of 8, against 4 for a simple alert policy and 0.2 on average for random selection.

HONEST NOTE

That demo proved the mechanism worked and little else. We had designed the questions and the scoring features together, on traffic we generated ourselves, so a good score was close to guaranteed.

12.3 Stage 3: the October version on real traffic

Next came a much larger version using five real public captures, 1,295 trials and an independent audit. The audit added a strong, simple comparator that used no weights at all. The result was uncomfortable:

- **A simple method beat the weighted one on several captures.** On the DonBot capture it scored 60.20% against 38.68%. The scoring was not earning its complexity.
- **Scan traffic used up the budget**, because all sessions were treated as equally important.
- **It could not scale.** Every packet was held in Python memory. At a 50% budget, a 58 MB capture took 230 seconds, so a multi-gigabyte run was out of reach.

BIG IDEA

This was the turning point. The October version was honest work, and its own audit showed it had to be replaced. EBR is a redesign, not a patch.

12.4 Stage 4: EBR

EBR kept the main lesson of the earlier work, that the unit of evidence is a conversation, and changed nearly everything else: streaming so it can handle gigabytes, value measured in bits, partial keeping of flows, a real optimisation, and a real IDS as judge. The evaluation also changed, from questions we wrote ourselves to detections by software we did not write.

12.5 Stages 5 to 8: three sealed tests

Part V tells this story in detail. In short, each test found something the previous one had missed, and the design changed to respond. That is how the work is supposed to go.

CHAPTER IN SHORT

The project moved from a teaching tool, to a synthetic demo, to a real-data audit that exposed its own flaws, to EBR. Each failure pointed to the next design.

Part IV

How EBR works

The algorithm, one step at a time. Each step is explained in plain words, then with a small calculation using the project's real code.

Tiers · Bits of surprise · Automation and pivoting · Diminishing returns · The knapsack · The ledger · Verify before delete

13 The idea in one picture

EBR asks one question of every byte: given a fixed storage budget, would an investigator regret losing this?

13.1 The pipeline

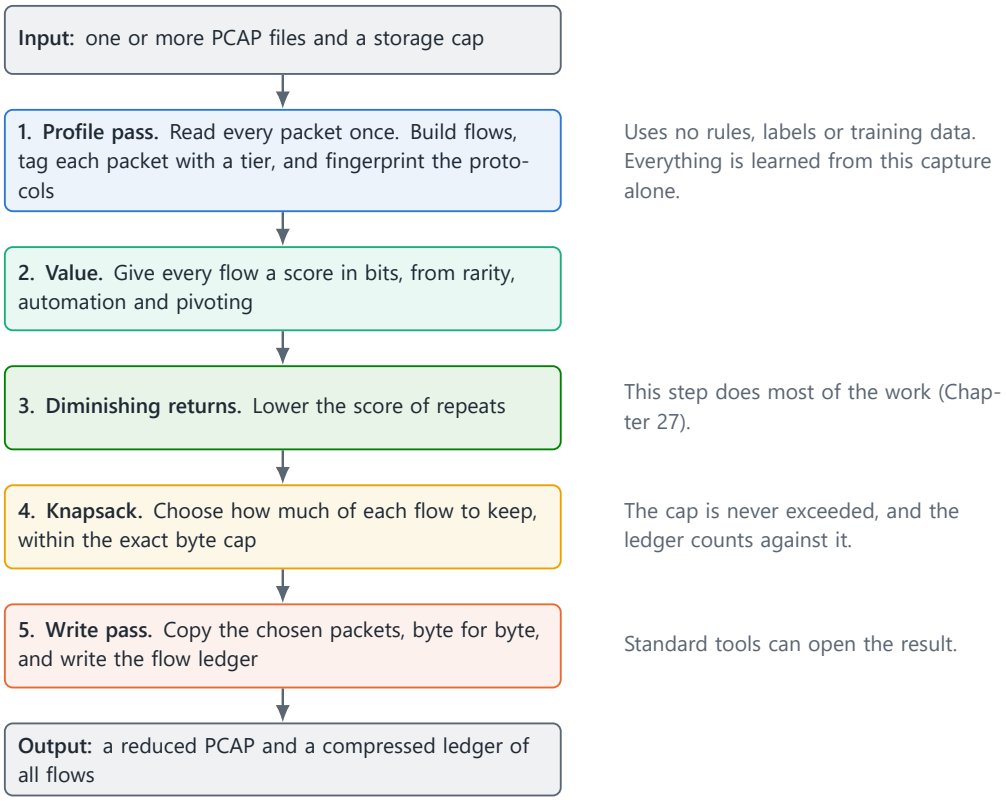


Figure 13.1. The EBR pipeline. Chapter 20 adds an optional checking loop around it.

13.2 Why two passes?

The October version held every packet in memory, which is why it could not scale. EBR **streams**: it reads a packet, notes a few numbers about it, and moves on. Packet contents do not stay in memory. Only the summary of each flow and a little information per packet remain.

The first pass decides. The second pass simply copies the chosen packets from the original file to the new one. With this design, the whole 11.3 GB first test was profiled and scored in about 231 seconds of processing time, roughly 20 seconds per gigabyte.

13.3 What makes it “evidence based”?

Each byte is kept according to how much evidence it adds. That evidence is measured in bits of surprise, then discounted for repetition. The next six chapters take these ideas one at a time.

BIG IDEA · EBR’s value formula

For each flow:

$$\text{value} = \big(1 + \max(\text{rarity},\ \text{automation}) + \text{pivot}\big) \times \text{repetition discount}$$

All parts are in bits, so they can be added without inventing arbitrary weights between them.

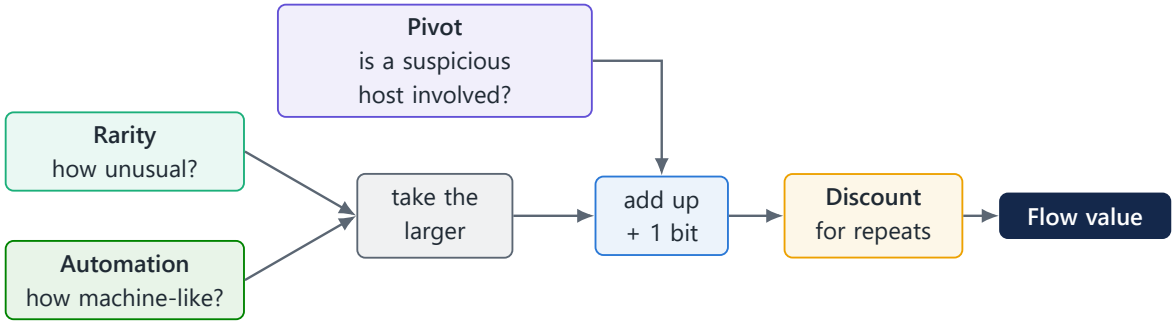


Figure 13.2. How the parts of the score combine.

14 Step 1: keep part of a flow

Older methods keep or drop a whole packet, or a whole flow. EBR's first idea is a middle ground: every flow can be kept at one of several levels of detail.

14.1 Five fidelity tiers

During the first pass, each packet gets a tier based on the role it plays in its flow.



Figure 14.1. The five tiers. Choosing a level for a flow keeps that tier and every lower tier.

Table 14.1. What each tier holds.

Tier	Name	Contents	Why it matters
0	Skeleton	SYN, FIN, RST, first and last packet	Proves the connection happened, and when
1	Head	First 1 KB each way	Requests, banners and response headers
2	Deep head	First 16 KB each way	Longer headers and the start of transfers
3	Turn heads	First 1 KB of each later message	Commands sent late in a long session
4	Bulk	Everything else	File bodies and encrypted data

UDP has no sequence numbers, so its tiers go by packet count in each direction: the first packet is the skeleton, packets up to the 8th are the head, packets up to the 64th are the deep head, and the rest are bulk.

14.2 Turn heads: finding commands late in a session

A plain cutoff keeps the start of a connection and nothing else. Imagine an attacker with a remote shell who types the important command an hour in. A cutoff has long since stopped recording.

A conversation goes back and forth, and each change of speaker starts a new **turn**. EBR's Tier 3 keeps the first kilobyte of every turn.

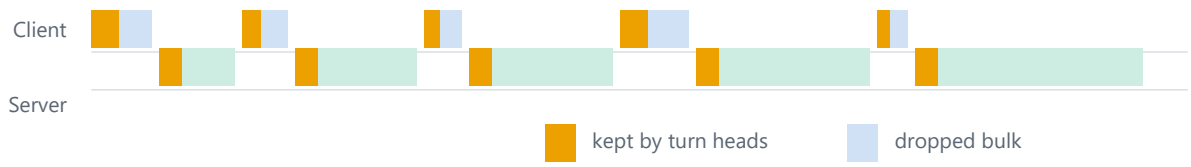


Figure 14.2. A long session in time order. Each message keeps its opening bytes, so the fiftieth command is still visible.

14.3 Two details that keep evidence usable

Positions come from TCP sequence numbers. A retransmitted packet lands in the same tier as the data it repeats, so a resend cannot sneak in or drop out by accident.

Pure ACKs follow the data they acknowledge. An ACK packet carries no data of its own, but an IDS that rebuilds conversations needs to see it. So each ACK takes on the tier of the data it confirms, which keeps the kept stream readable by Suricata.

> **CHAPTER IN SHORT**
>
> A flow can be kept at five levels of detail, from a bare skeleton to everything. This lets EBR spend bytes in small steps, instead of making an all-or-nothing choice.

15 Step 2: measuring surprise in bits

EBR needs a single number for how interesting a flow is. It borrows that number from information theory, which started with Claude Shannon in 1948 [1].

15.1 Surprise, in plain words

If your friend tells you the sun rose this morning, you learn nothing. If they tell you it snowed in the desert, you learn a lot. **Rare events carry more information.**

The information in an event with probability P is measured in **bits**:

$$\text{surprise} = -\log_2 P$$

One bit is the information in one fair coin flip. The formula counts how many coin flips in a row would be as unlikely as this event.

Table 15.1. Probability and surprise.

How often it happens	Surprise	As unlikely as
1 in 2	1 bit	1 head
1 in 8	3 bits	3 heads in a row
1 in 1,024	10 bits	10 heads in a row
1 in 65,536	16 bits	16 heads in a row

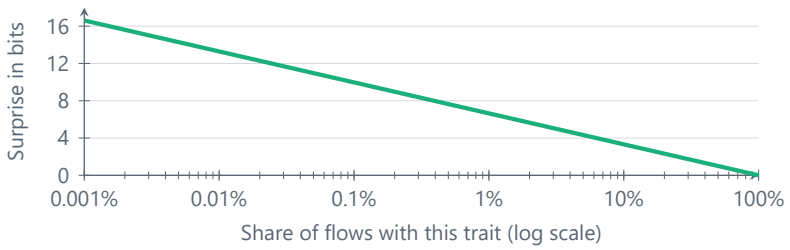


Figure 15.1. The rarer a trait, the more bits it is worth. A trait on half of all flows is worth just 1 bit.

15.2 Five traits of a flow

EBR measures surprise on five traits, using the capture itself as the baseline.

Table 15.2. The five traits EBR measures.

Trait	What it is	Example of a surprise
Service	Protocol, port and application	A file-sharing service seen once
Flow shape	Application, how it ended, sizes each way	A web flow with a huge upload
Client–server pair	Who talked to whom	Two machines that never otherwise talk
Client fingerprint	HTTP header layout and User-Agent, a hash of the TLS ClientHello, or the SSH banner	Software nobody else on the network uses
Content skeleton	The structure of the request, with the words removed	An odd-looking web address

The content skeleton is a neat trick. It keeps punctuation and removes words:

```
/page?id=7                 ->  /a?a=a
/page?id=9                 ->  /a?a=a            (the same skeleton: boring)
/page?id=1'%20OR%201=1--   ->  /a?a=a'%a%a=a--   (a different skeleton: surprising)
```

The third line is the SQL injection from Chapter 7, with spaces encoded as `%20`, as in a real web address. EBR has no idea what SQL injection is. The attack has an unusual shape, and that is enough to raise its value.

15.3 Combining the traits

For each trait, EBR first subtracts the typical (median) surprise, because only surprise above normal counts. It then lets the most surprising trait dominate and adds a quarter of the rest:

$$\text{rarity} = \text{largest excess} + 0.25 \times (\text{sum of the other excesses})$$

Each trait is capped at 16 bits, so one extreme value cannot run away with the score.

15.4 The Good–Turing correction

There is a trap. Suppose a spam bot looks up thousands of different mail servers. Every name appears once, so every name looks extremely rare, and plain counting would give each of them a huge score. The budget would disappear into junk.

The problem is that when almost everything is new, one more new thing is not surprising. Alan Turing and I. J. Good worked out how to handle this [2]: the share of values seen exactly once estimates the chance that the next value will be new. EBR calls this share the **novelty**, and it estimates probability as:

$$P(\text{value seen } n \text{ times out of } N) = \max\left(\frac{n}{N},\ \frac{\text{novelty}}{n}\right)$$

WORKED EXAMPLE · Real output from the project's code, 100 flows

Case A: one-offs are rare. 99 flows share one DNS name, and 1 flow has a unique name. Novelty is 1%. The unique name scores **6.64 bits**.

Case B: one-offs are common. 50 flows share one name, and 50 flows each have a unique name. Novelty is 50%. Each unique name now scores just **1.00 bit**. Plain counting would have given it 6.64 bits.

So a one-off only scores highly when one-offs are rare.

CHAPTER IN SHORT

Rarity is measured in bits, using the capture itself as the baseline. Five traits are combined. The Good–Turing correction stops floods of unique junk from looking interesting.

16 Step 3: automation and pivoting

Rarity alone has two blind spots. A flood is not rare, because its traits become common. A quiet flow from a bad host looks harmless on its own. Two more signals cover these gaps.

16.1 Automation: when many flows look machine-made

A person does not send 10,000 near-identical connections. For each group of flows sharing a client, server, port and protocol, with at least 8 flows, EBR computes:

$$\text{automation} = \log_2(n) \times \text{likeness}$$

Here n is the number of flows, and **likeness** (0 to 1) is the strongest of three clues that the group is machine-made:

- **Same sizes:** the flows are nearly identical in size.
- **Regular timing:** the gaps between flows are even, like a clock.
- **Failures:** a large share of the flows failed (S0, REJ or RSTO).

> WORKED EXAMPLE · A brute-force group
>
> 1,024 login attempts from one client to one server, nearly all the same size, gives likeness close to 1 and automation of about $\log_2(1024) \times 1 = 10$ **bits**. That is as much as a 1-in-1,024 rare event, even though each attempt looks common.

Scanners get the same treatment. A client whose failed attempts reach many different endpoints scores $\log_2$ of the number of endpoints, even if each endpoint saw only one flow.

16.2 Pivoting: spreading suspicion like an investigator

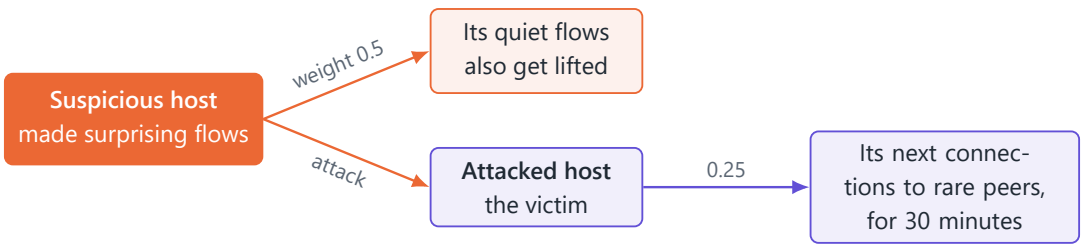


Figure 16.1. Pivoting. Suspicion spreads to everything a suspicious host touched, and to what its victims did next.

An investigator who distrusts a host looks at everything it touched, so EBR does too, in two ways:

1. Suspicion spreads from a host to all of its flows, with half weight. Quiet flows from a bad host get a lift.

2. If a host was attacked, its next connections to rarely seen peers are lifted for 30 minutes, with quarter weight. That is where a compromise shows up.

A host's suspicion is the *total* surprise it initiated, not its single strangest flow, so one odd connection does not condemn it. Only flows above 6 bits count, and each host's score is measured relative to the typical host.

> **HONEST NOTE**
>
> Testing later showed that pivoting made no measurable difference to signature recall on these captures (Chapter 27). It is sensible, and it produces the useful "suspect list", but we cannot claim it improves retention.

> **CHAPTER IN SHORT**
>
> Automation catches floods, using $\log_2 n$ scaled by how machine-like a group looks. Pivoting copies the way investigators follow a suspicious host.

17 Step 4: diminishing returns

This is the most important step in EBR. Testing showed that it does more than any other part.

17.1 The key insight

The first failed login tells you an attack is happening. The ten-thousandth tells you almost nothing new, provided you still know the count. So the value of each extra copy of a behaviour should fall.

> EVERYDAY PICTURE
>
> A photo album of a holiday needs one good picture of the beach. Fifty near-identical shots add little, and a note saying "we took fifty" covers the rest. But a photo of a different beach is new, even if it looks similar.

17.2 Two strengths

That analogy explains why EBR uses two rates of decline.

- **Repeats fall steeply.** The same message, from the same client, to the same server. The discount is $1/\sqrt{r}$.
- **Variations fall gently.** New content or a new endpoint may be a new indicator of compromise, so it keeps most of its value. The discount is $1/\sqrt[4]{r}$.

Here r is the flow's place in the line once the first 16 flows have been set aside, which the next section explains. The 17th flow has $r = 2$, the 18th has $r = 3$, and so on.

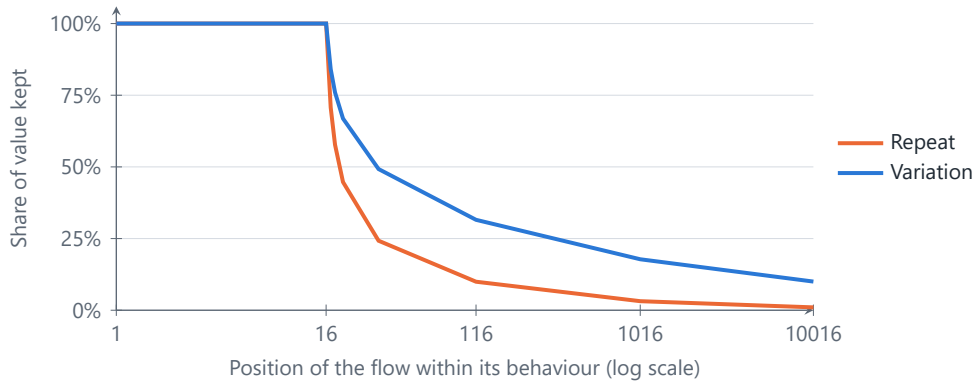


Figure 17.1. The real discount curves. The first 16 flows keep full value, then value falls. By flow 10,016, a repeat is worth 1% and a variation 10%.

17.3 How behaviours are grouped

EBR groups each flow in several ways, so that floods collapse from any direction:

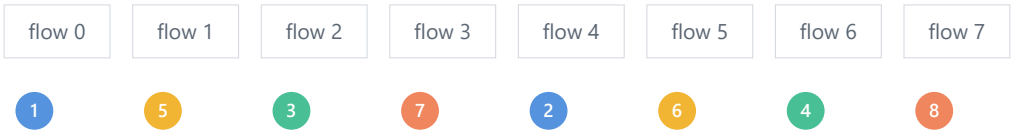
- **Exact repeat:** same client, server, service, content and shape.
- **Same client:** same client and flow shape. Leaving the port out makes a thousand-port scan count as one behaviour.
- **Same server:** same server, service and shape. This collapses many clients hitting one target.
- **Silent flows:** a flow with no payload holds nothing the ledger lacks, so it falls at the steep rate too.

17.4 Exemplar bursts

The first 16 flows of each behaviour keep their full value. This is deliberate. Rate-based IDS rules need several events close together, and spreading out single samples would never trigger them. These first flows are called **exemplars**.

17.5 Bit-reversed order

After the first 16, which flows should be favoured? If value simply fell with time, you would keep the start of a long attack and lose its end. EBR visits the remaining flows in **bit-reversed order**, so that any prefix of the list covers the whole time span.



Visiting order 0, 4, 2, 6, 1, 5, 3, 7. After 2 picks the capture is split in half, after 4 in quarters.

Figure 17.2. Bit-reversed order for 8 flows, taken from the real code. Numbers in circles show the order of visiting. Evidence stays spread across time however early the budget runs out.

> **BE CAREFUL · The blind spot**
>
> Two login attempts with different passwords have the same size and shape. For a Telnet login, EBR's content summary looks only at the shape of the opening bytes, so the password never reaches it. As a result, a password spray looks like pure repetition, and most of it is discarded. The third sealed test exposed this, as Chapter 26 explains.

> **CHAPTER IN SHORT**
>
> Repeats lose value quickly and variations slowly. The first 16 of any behaviour keep full value for rate-based rules. Bit-reversed order keeps evidence spread across time.

18 Step 5: the knapsack

Every flow now has a value. EBR still has to decide how much of each flow to keep. This is a classic problem in computer science.

18.1 The problem

> EVERYDAY PICTURE
>
> You are packing a bag with a weight limit. Each item is worth something and weighs something, and you want the most value in the bag. Now add a twist: each item comes in versions, such as a thin book, a thick book or the full encyclopaedia, and you may pack only one version of each.

That twisted version is the **multiple-choice knapsack problem** [3]. Here, the items are flows, the versions are fidelity levels, the weight is bytes, and the limit is the cap. Solving it perfectly is very hard at this scale. Happily, a simple greedy method comes very close.

18.2 How much is each tier worth?

Each tier is worth a share of the flow's value. The head is the most valuable by far.

Table 18.1. The share of a flow's value earned by each tier.

	Skeleton	Head	Deep head	Turn heads	Bulk
Readable flows	20%	50%	15%	10%	5%
Encrypted flows	20%	50%	1.5%	1%	0.5%

For flows that nobody can read, everything past the head is worth a tenth as much, because extra encrypted bytes reveal little. SMB counts as readable, since it is unencrypted and is where human attackers move.

A flow earns value only for evidence it actually contains. A bare SYN has no head to keep, so it cannot outbid a real request just by being cheap.

18.3 The greedy method

1. For each flow, work out its levels: the bytes they cost and the value they earn.
2. Remove any level that is poor value compared with its neighbours. What is left is called the **upper concave hull**: each step up gives less value per byte than the one before.
3. Put every step from every flow into one list, sorted by value per byte.
4. Take steps from the top until the budget runs out.

WORKED EXAMPLE · Real numbers from the project's code

Take two flows with a value of 10 and the same tier sizes: 300, 1,800, 6,000, 2,000 and 40,000 bytes. Flow A is readable HTTP. Flow B is encrypted TLS.

Step	Flow	Bytes added	Value per byte
1	A and B: skeleton	300 each	0.00667
2	A and B: head	1,800 each	0.00278
3	A: deep head and turns together	8,000	0.00031
4	B: deep head and turns together	8,000	0.00003
5	A: bulk	40,000	0.00001

With 13,000 bytes to spend, EBR takes steps 1, 2 and 3. Flow A keeps 10,100 bytes and Flow B keeps 2,100. Both had equal value, but the readable flow gets the extra budget because its bytes are worth ten times more. Tier 2 merges with Tier 3 because, alone, it is poorer value than the step after it.

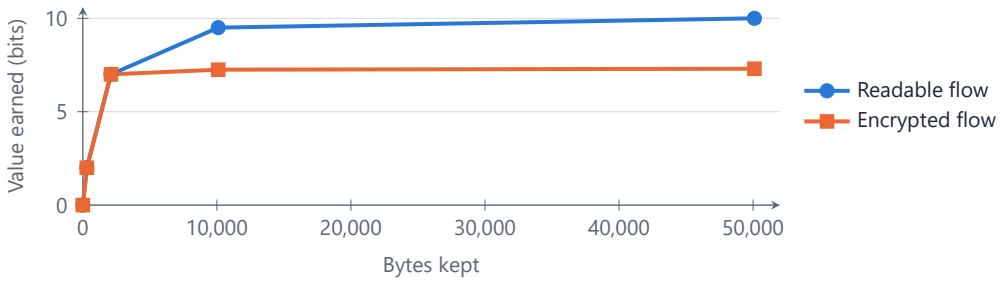


Figure 18.1. The concave hull for the two example flows. The curve bends over quickly: the first few kilobytes earn most of the value.

18.4 The scan cap

Flows with no payload are so cheap that a scan storm can outbid real evidence on price alone. On one red-team capture, they took 186 MB of a 227 MB packet budget. So payload-free flows may use at most **30%** of the budget at first. Anything that real content leaves unused goes back to them afterwards.

HONEST NOTE

With the cap added, the problem gains a second constraint, so the result becomes a good approximation rather than the fractional best. The cap also failed its own pre-registered test, so it is a safeguard, not a proven gain (Chapter 26).

CHAPTER IN SHORT

EBR picks one level per flow. It sorts all possible steps by value per byte and takes them until the cap is reached. Readable bytes win over encrypted ones. A 30% share stops scan storms from crowding out content.

19 Step 6: the flow ledger and the output

Whatever happens to a flow's packets, EBR never forgets that the flow existed.

19.1 The ledger

EBR writes a small compressed table, the **flow ledger**, with one row per flow. Even a flow with no surviving packets is recorded.

```
start_ns, seconds, proto, client, cport, server, sport, app, state, pkts_c2s, pkts_s2c, ...
```

This answers questions that packets alone cannot, once most of them are gone. How many login attempts were there? When did the scan start and stop? Which hosts took part?

> WORKED EXAMPLE · Why the ledger matters
>
> In the first sealed test at a 5% cap, the ledger gave the attacker's exact flow count in all five labelled cases, including all 1,908,929 flows in the largest, with first and last times correct to within a millisecond.

19.2 Folding, when space is tight

When the budget is small, even the ledger must shrink. Its share is capped at 10% of the budget. Low-value flows are then folded into grouped rows that still keep exact totals: how many flows, when they ran, how many bytes. Individual details are lost, but the counts stay correct.

19.3 The ledger pays its way

> BIG IDEA
>
> The ledger is charged against the same byte budget as the packets. If it were free, EBR would get extra storage that other methods do not get, and the comparison would be unfair.

The ablation test showed the cost: removing the ledger raised the attacks caught at a 1% cap from 5 to 7. That is the price of exact accounting, and a real trade-off.

19.4 The reduced capture

In the second pass, EBR copies every chosen packet into a new PCAP file, byte for byte. It changes nothing and invents nothing. It also records SHA-256 hashes, so anyone can check the files later.

19.5 A real example

Here is one real case from the demonstration at a 5% cap, a day of denial-of-service attacks from the CSE-CIC-IDS2018 dataset.

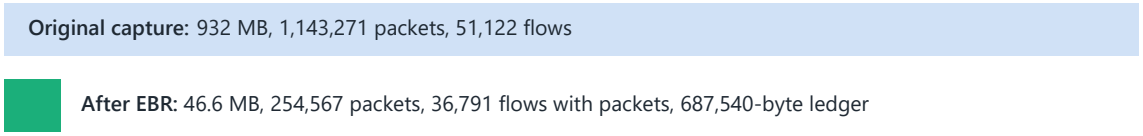


Figure 19.1. Bars are drawn to scale. The reduced file holds 5% of the bytes but 22% of the packets, because EBR drops bulk and keeps small packets.

- It kept 72% of all flows with at least some packets, and the ledger accounts for the rest.
- EBR's suspect list put one of the two labelled attackers, `18.219.211.138`, first of 1,964 hosts. The other attacker ranked 167th, so the list is a helpful starting point and not a verdict.
- Processing this file took about 5.6 seconds.

> **CHAPTER IN SHORT**
>
> The ledger records every flow, folds when space is tight, and is paid for out of the budget. The reduced PCAP contains only exact copies of original packets.

20 Verify before delete

This is the project's most successful idea, and it came from questioning an assumption that everyone in this field makes.

20.1 The assumption

Selective retention is normally a blind bet: choose what to keep, delete the rest, and hope. Nobody finds out what was lost until an investigation fails.

> **BIG IDEA**
>
> The original capture still exists until the moment you delete it. So before deleting, you can measure what the reduced copy lost, and fix it.

20.2 The loop

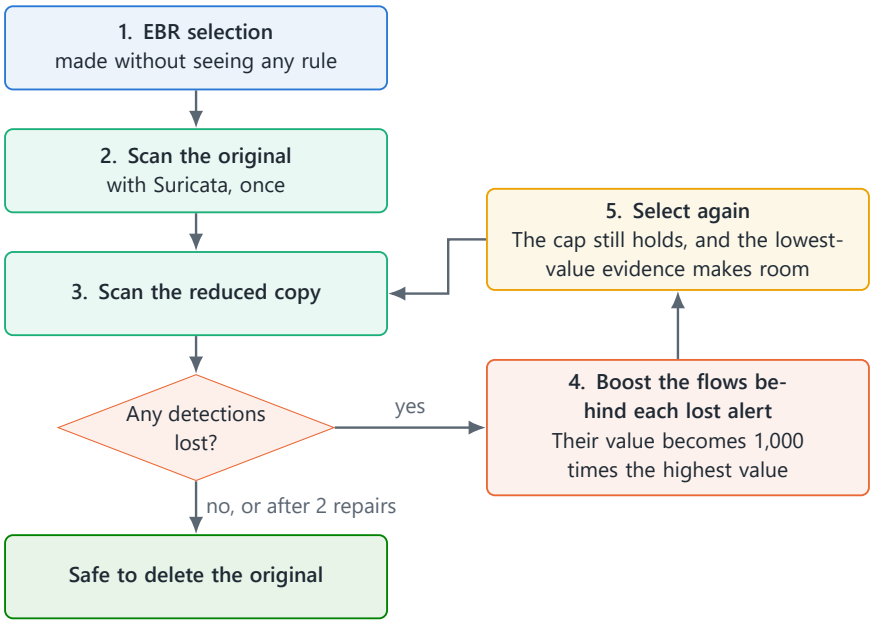


Figure 20.1. The verify-before-delete loop. There are at most two repair rounds.

20.3 The details

First try. EBR restores the flow that raised each lost alert, found by its five-tuple.

Second try. If an alert is still lost after its own flow came back, it is probably a rate-based rule, which needs many flows. The next round adds up to 64 neighbouring flows between the same two hosts, within the same minutes as the alert.

The cap never moves. A repaired flow simply outbids everything else, and the lowest-value evidence is pushed out.

20.4 Does this not defeat the point?

A fair objection is that EBR was meant to avoid depending on an IDS. The answer has two parts.

- The first selection is still free of rules, so protection against unknown threats still comes from EBR alone.
- The IDS only repairs what it can already see. It runs once per batch, not live, so it does not steer what gets captured.

20.5 The costs

Table 20.1. What verification costs.

Cost	Measured
Scanning time	The original must be scanned before deletion, about 62 seconds per GB on our lab setup
Budget	Repaired flows used 15.6%, 13.1%, 5.0% and 1.4% of the budget at the 1, 2, 5 and 10% caps in the third test
Some cases get worse	3 of 64 case comparisons fell in the third test, because repaired flows displace other evidence
Some alerts stay lost	249 of 2,041 usable alerts at 5%, mostly rate-based scan rules
Unknown threats	Repair only protects what a rule can already see

CHAPTER IN SHORT

Do not delete blindly. Scan both copies, restore what was lost, and only then delete. This stays within the same budget and was the strongest idea in the project.

Part V

The tests

How the experiments were built so that a good score would mean something, and what each one found, including what went wrong.

Fair testing · The lab and the data · Three sealed tests · Which parts matter · Every mistake we made

21 How to test fairly

It is easy to fool yourself in this kind of work. You can tune an algorithm until it fits your test data and call that a discovery. Each rule below blocks one way of fooling yourself.

21.1 The basic experiment

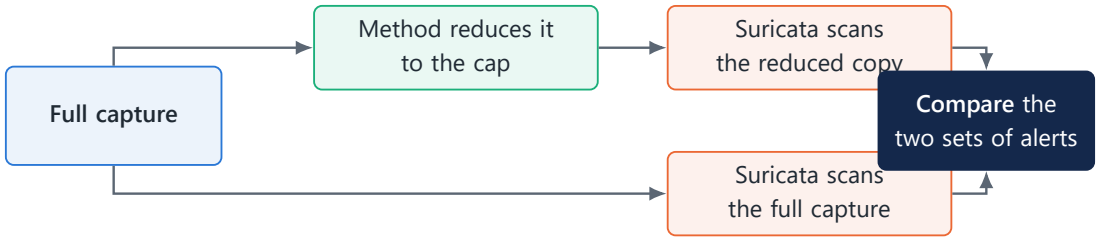


Figure 21.1. Each method is scored by how many of the full capture's detections survive in its reduced copy.

21.2 What we measure

> NEW WORDS
>
> **Recall** is the share of something that survived. If the full capture raises 30 attack signatures and the reduced copy still raises 27 of them, recall is $27 \div 30 = 90\%$.

Table 21.1. The measures used in this project.

Measure	What it counts	Why it is useful
Attack signature recall	Distinct attack-category rules that still fire	Primary measure. Did each *kind* of attack stay visible?
Alert instance recall	Distinct combinations of rule and host pair that still fire	Finer detail. Did it survive on each pair of hosts?
Later-rule recall	Alert instances from rules published on or after the capture date	The time-travel measure. Nothing could have used these rules at capture time
Labelled attacks caught	Attacks named by the dataset publisher that still link attacker to victim	Ground truth from a third party

We report signature recall first because it is the steadier measure, for a reason Chapter 26 explains: scanning the same file twice does not always give the same instance count.

A case only counts if the full capture raises at least one attack alert. If nothing fires on the complete recording, there is nothing to preserve, and the case is **not scored**.

21.3 Rule 1: an independent judge

The judge is Suricata with 52,778 public rules. We did not write it and EBR never sees those rules. Scoring EBR by what it was built to keep would be meaningless.

21.4 Rule 2: independent ground truth

Attack times and addresses come from the dataset publishers, not from us.

21.5 Rule 3: strong, fair comparators

- The flow cutoff is fitted to every budget, which makes it stronger than a fixed setting.
- The IDS-triggered method receives real alerts from rules older than the capture. EBR gets no rules at all.
- The cap counts every serialised byte, and EBR's ledger comes out of its own allowance.

21.6 Rule 4: separate development and test data

Tuning happens on **development cases**. Final results come from **test cases** that were never opened during tuning. Using test data for tuning is like seeing the exam paper before the exam.

21.7 Rule 5: seal it

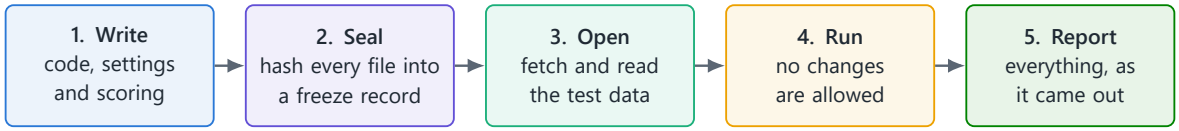


Figure 21.2. A sealed test. Anyone can recompute the hashes to confirm nothing changed after step 2.

The freeze record for the third test covers 22 files, including the plan itself. Running `python -B final/freeze.py check v3` recomputes every hash.

21.8 Rule 6: pre-register

Sealing stops you changing the method. **Pre-registration** stops you changing the question [15]. Before opening the third test's data, we wrote down five hypotheses, the primary measure, and the exact rule for calling each one supported. Without this, it is tempting to look at the results and decide afterwards what you were really testing.

21.9 Rule 7: use honest statistics

With only a few cases, one lucky case can swing an average. The **bootstrap** [4] shows how much the average could plausibly vary.

WORKED EXAMPLE · The bootstrap, step by step

Say a method beat another by these points on 8 cases.

1. Draw 8 cases at random from that list, allowing the same case to be picked twice.
2. Compute the mean difference for that draw.
3. Repeat 10,000 times.
4. Sort the 10,000 means. The middle 95% of them is the **95% interval**.

If that interval does not include zero, the difference is unlikely to be a fluke of which cases we happened to have. That was the pass rule for every hypothesis.

CHAPTER IN SHORT

A fair test uses an outside judge, outside ground truth, strong comparators, separate tuning data, sealed code, predictions written in advance, and intervals that respect how few cases there are.

22 The lab and the data

Where the tests ran, and what they ran on.

22.1 The laboratory

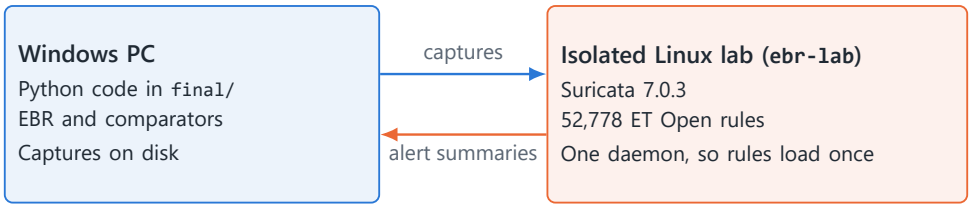


Figure 22.1. The lab. Captures are only read from files. Nothing is ever replayed onto a real network.

Suricata runs in a dedicated Linux environment under WSL 2 on the Windows machine. Each run produces a compact summary of its alerts. Thousands of scans were needed, so one background daemon keeps the rules loaded, which saves a long start-up for each file.

22.2 The datasets

Table 22.1. The data sources.

Dataset	What it is	Strength and weakness
CSE-CIC-IDS2018 [16]	A large simulated company under scripted attacks, by day	Labelled attackers and times. Attackers are easy to separate, and such lab datasets have known flaws [17]
WRCCDC	Recordings from a student cyber-defence competition with a live red team	Real humans improvising. No exact labels
CTU-Mixed [18]	Real captures with live malware infections	Real malware traffic
IoT-23 [19]	Recordings of infected internet-connected devices	Enormous scan volumes. Dominated by one infected host

22.3 The three test sets

Table 22.2. What each sealed test contained.

Test	Size	Packets	Scored	Contents
First	11.28 GB	35.0 M	6 of 6	Five days of CIC-IDS2018, one WRCCDC 2018 red-team slice
Second	7.03 GB	15.1 M	5 of 6	WRCCDC 2019, 2024 and 2026, two CTU networks, one CIC DDoS day
Third	6.15 GB	34.0 M	8 of 9	Six IoT-23 captures, three WRCCDC slices from 2017 and 2018

Each test used captures that had not been seen before. One disclosure: the WRCCDC 2018 slices in the third test come from the same event as a capture used in development, at different hours. They are unseen data, not a new network.

22.4 How the runs were kept honest

- **One crash, disclosed.** The first frozen run stopped before anything was scored, because one 4.3 GB capture was PCAPNG. It was converted to classic PCAP, the independent tool Scapy confirmed all 18,890,171 packets matched with zero mismatches, and no frozen file changed.
- **Replay checks.** The first test can be rerun from its frozen settings to confirm that the selections match.
- **Scan noise is measured.** See Chapter 26.

23 The first sealed test

11.3 GB, 35.0 million packets, six cases. This tested the first version of EBR against everything else, with no rules and no second chances.

23.1 Results

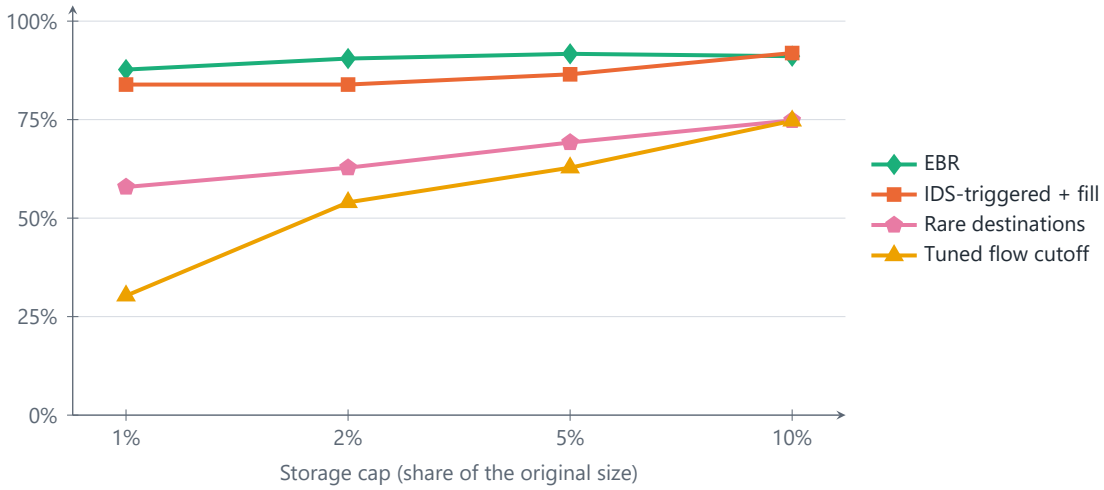


Figure 23.1. First sealed test: attack signatures preserved, mean across cases.

Table 23.1. Labelled attacks still caught, out of the 8 that Suricata detects on the full capture.

Method	1%	2%	5%	10%
EBR	5	7	**8**	**8**
IDS-triggered + fill	7	7	7	8
Tuned flow cutoff	0	2	5	7
Rare destinations	3	3	3	4
Random flows (mean of 3 seeds)	5.7	6.3	6.3	6.3
Random packets (mean of 3 seeds)	0	0.7	1.7	3

23.2 What it showed

- **EBR beat the methods that use no rules.** Against the tuned cutoff and rare destinations, the 95% interval on its advantage excluded zero at every cap.
- **It was level with an IDS given real rules.** EBR was ahead at 1–5% and behind at 10%, but every interval included zero, so this test cannot separate them. That is still a strong result for a method that used no rules.
- **The ledger was accurate.** It gave the attacker's exact flow count in all five labelled cases at a 5% cap.

23.3 What did not go EBR's way

- **Tight caps favour an IDS on alert instances.** At 1%, IDS-triggered capture kept 91.7% of alert instances against EBR's 49.2%.
- **The time-travel prediction failed.** We expected later-published rules to favour EBR. IDS-triggered capture still kept 92.9% of those alerts against 85.0% at 5%, because new rules mostly fire on flows that old rules had already flagged.
- **Human red-team traffic was the weak spot.** On WRCCDC, EBR kept 60.7% of attack signatures at 5%, against 78.6% for IDS-triggered capture. Of the 11 signatures it never recovered, 10 were SMB lateral movement or SMB exploits.
- **More budget sometimes lost detections.** Three WRCCDC signatures present at a lower cap were gone at 10%.
- **The suspect list was uneven.** The attacker ranked #1, #1 and #7 in three labelled cases, and did not appear at all for the bot and infiltration cases.

> **BIG IDEA**
>
> A single good number hides a lot. Overall the mean looked strong, but one case exposed a real flaw. Looking case by case is what led to the next version.

24 The second sealed test

7.0 GB of fresh captures, testing a fixed version. It went well, but it had an unexpected catch.

24.1 The diagnosis and the fix

Why was EBR weak on the red-team capture? The first explanation was gaps inside flows. An experiment disproved that: the affected login flows alert at every fidelity level when scanned alone. Checking where the bytes went gave the real answer: **scan flows took 82% of the budget**.

A steeper discount looked like the fix until it was tried on all 13 earlier cases, where it damaged others. The final changes were the 30% scan cap (Section 18.4) and treating SMB as readable.

24.2 Results

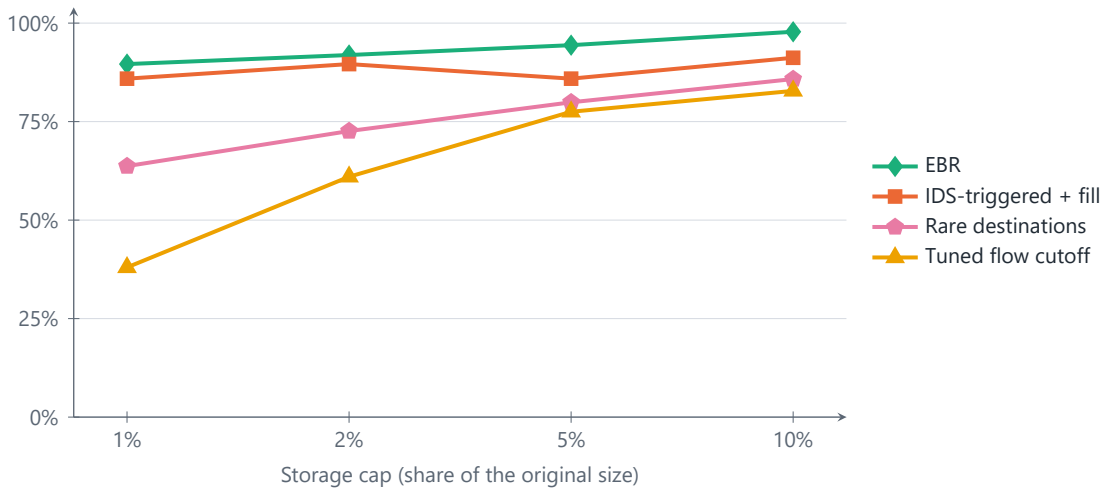


Figure 24.1. Second sealed test: attack signatures preserved, mean across five scored cases.

- **EBR led every method on signature recall at every cap.**
- Its lead over rare destinations cleared the 95% interval at every cap.
- Its lead over IDS-triggered capture cleared the interval at 5% and 10% (+8.5 and +6.5 points), but not at 1% and 2%.
- On alert instances the two were closer. IDS-triggered capture kept more at 1% and 2%, and EBR kept more at 5% and 10%. None of those differences cleared the interval.

24.3 The catch

HONEST NOTE

This test did not exercise the fix at all. Payload-free flows would have taken at most 3.4% of the budget on these captures, against 82% on the capture that exposed the flaw. The scan cap never engaged, and the old and new versions tied on signature recall in every case. The evidence for the cap at this point was therefore not sealed. That is why a third test was needed, chosen specifically to be scan-heavy.

25 The verify-before-delete prototype

The first trial of the repair loop, run on captures whose results we already knew. Keep that in mind: this is a promising prototype, not a sealed result.

> **BE CAREFUL**
>
> Nothing in this chapter is sealed. The loop's settings were fixed before the run and not tuned afterwards, but the outcomes of these captures were already known.

Table 25.1. First results of the repair loop, across 11 cases, using only rules older than each capture.

Measure	Method	1%	2%	5%	10%
Attack signatures	EBR with repair	**95.7**	**98.7**	**99.6**	**100**
	EBR alone	88.6	91.8	96.2	98.7
	IDS-triggered + fill	84.8	86.5	86.2	91.6
Alert instances	EBR with repair	68.7	84.6	**97.2**	**97.1**
	EBR alone	59.8	76.8	90.2	92.9
	IDS-triggered + fill	**85.1**	**87.7**	85.1	89.3

With repair, EBR beat IDS-triggered capture on signature recall at every cap, by 8 to 13 points, and every 95% interval stayed above zero. Six of 88 case comparisons got worse, by up to 15 points at a 1% cap.

25.1 The blind-spot estimate

This prototype also tried an inventive idea. Hide half of the rule families from the repair step, and see how many hidden alerts EBR kept. That estimates how well EBR handles threats with no rule, and it could be computed daily. On this unsealed data it looked promising, with an average error of 9.1 points against 26.8 for guessing. It later failed, as the next chapter shows.

26 The third sealed test

The most important experiment in the project: 6.1 GB of captures chosen to be scan-heavy, with five written predictions, opened only after the plan was hashed.

26.1 The plan

Version 3 was frozen on 31 October 2026 at 00:54:12, with hashes of 22 files. A blind screen then read packet headers only, with no detection, to find which captures had scan storms. It found 7 of 9 scan-heavy, so unlike the second test, this one really exercised the scan cap.

Three versions were compared: EBR, the first version of EBR (with no scan cap), and EBR with repair.

26.2 Results

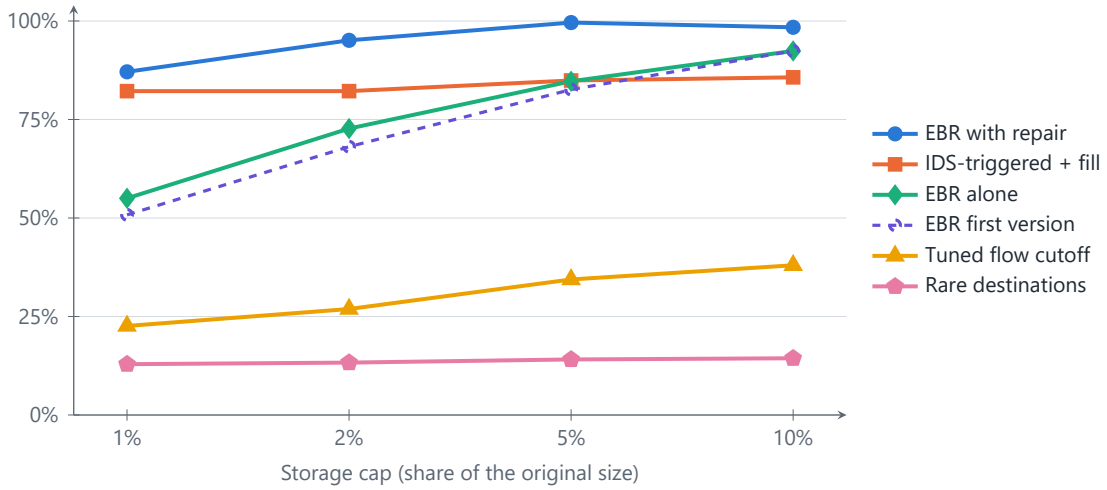


Figure 26.1. Third sealed test: attack signatures preserved, mean across 8 scored cases.

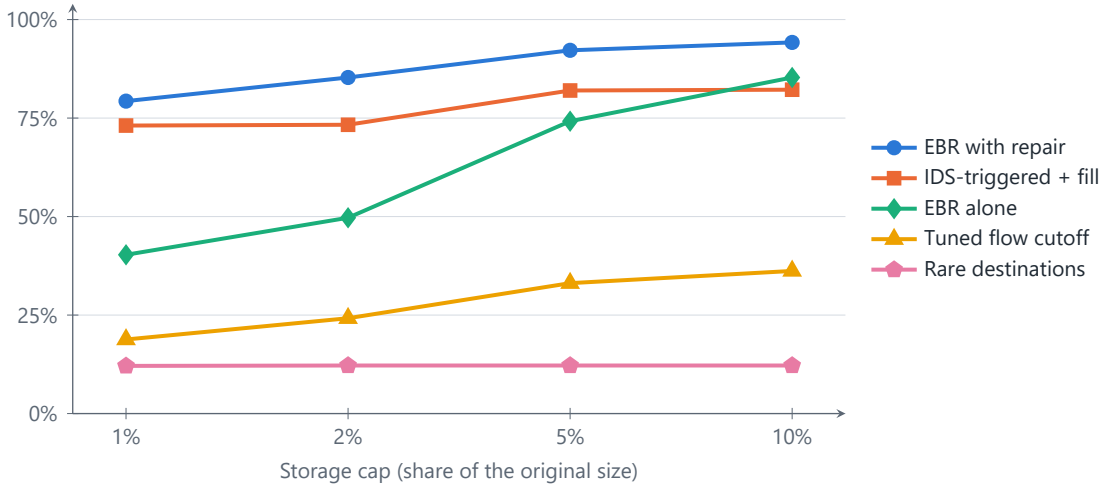


Figure 26.2. Third sealed test: attack alert instances preserved, a finer and noisier measure.

Table 26.1. Attack signatures kept at a 5% cap, case by case.

Case	EBR	First version	IDS + fill	EBR + repair
IoT-23 1-1 (Hide and Seek)	52%	52%	100%	**100%**
IoT-23 7-1	57%	57%	100%	**100%**
IoT-23 9-1	100%	100%	100%	**100%**
IoT-23 48-1	100%	100%	83%	**100%**
IoT-23 49-1	100%	100%	67%	**100%**
WRCCDC 2017 at 75%	88%	88%	100%	**100%**
WRCCDC 2018 at 25%	93%	77%	67%	**97%**
WRCCDC 2018 at 75%	88%	88%	62%	**100%**

26.3 The verdicts

Table 26.2. The five pre-registered hypotheses.

	Prediction	Verdict	Evidence (points)
H1	On scan-heavy cases, the scan cap beats the first version at 5% and 10%	**Failed**	+2.8 at 5%, and the interval reaches zero. 0.0 at 10%
H2	Repair beats EBR alone at every cap	**Passed**	+32.1, +22.4, +14.9, +7.6
H3	Repair beats IDS-triggered + fill at every cap	**Failed**	Clear at 2%, 5%, 10%. Inconclusive at 1%
H4	Repair costs later-rule recall no more than 5 points at any cap	**Failed**	−7.5 at 2%, from just four cases
H5	The blind-spot estimate beats guessing	**Unscoreable**	8 usable pairs of the 10 required. On those, 24.5 error against 14.6

The intervals tell the story better than the averages. In the charts below, a dot is the average gain and the bar is the 95% interval. A bar that crosses the zero line does not pass.

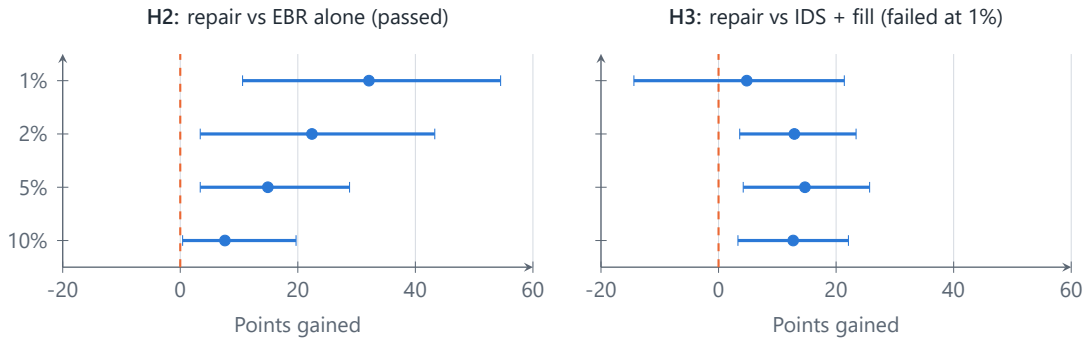


Figure 26.3. Paired differences by cap, with 95% intervals from resampling cases. The dashed line is zero.

26.4 Why it came out this way

The scan cap had little to protect

A botnet host makes millions of scan flows, but few flows have any content, and that content usually fits anyway. The cap helped in 3 of 8 cases and did not lower signature recall in any. It is a safeguard against a rare failure, not a general improvement.

Login sprays look like repeats

On the Hide and Seek capture, EBR alone lost 110 of 158 attack alert instances at a 10% cap, mostly Telnet logins with known default passwords. Each attempt has the same size and shape, and the difference lies in the password. Diminishing returns sees repetition and throws them away.

> **BIG IDEA · A real blind spot**
>
> This is a weakness in the very idea that does most of EBR's work. It explains why EBR alone did poorly at low caps here. The repair loop fixes it when a rule exists. Without a rule, EBR alone remains exposed.

Rate-based rules are hard to repair

At 5%, 249 of 2,041 usable alerts stayed lost, mostly scan rules that need many flows close together. Restoring those can cost more budget than exists.

26.5 Scan noise

The same selection, scanned twice by Suricata, does not always give the same answer. Across 32 comparisons, 12 differed, by 0.4 points of instance recall on average and up to 6.1 on one case. Signature recall moved by 0.4 points on average, and by 12.5 on one case with only 8 signatures. Running Suricata in single-threaded mode did not remove this, so differences of a point or two should be ignored.

26.6 What it costs

Repaired flows used 15.6%, 13.1%, 5.0% and 1.4% of the budget at the four caps. Repair made 3 of 64 case comparisons worse. Lossless gzip reached only 39.9% of the original size on these captures.

CHAPTER IN SHORT

Repair is the clear winner, passing H2 at every cap and beating IDS-triggered capture at 2%, 5% and 10%. The other predictions did not pass in full. The scan cap is a safeguard, and the blind-spot estimate is an open idea. EBR alone is not always ahead, because traffic type matters.

27 Which parts of EBR matter?

An algorithm with many parts raises a fair question: are all of them doing anything? We found out by removing them one at a time. This is called an ablation study.

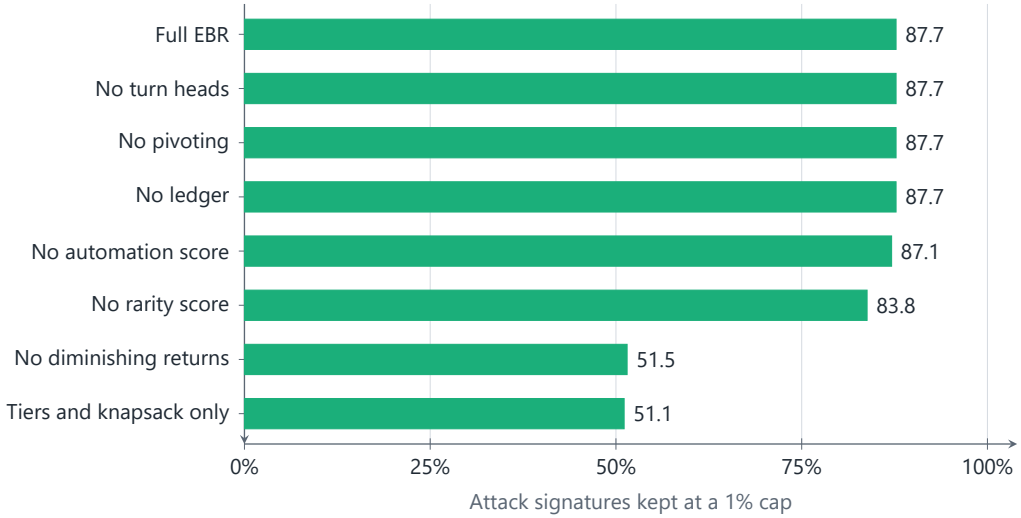


Figure 27.1. Removing each part in turn, first sealed test data, 1% cap. Only diminishing returns makes a large difference on its own.

Table 27.1. Full ablation results, attack signature recall, first test data.

Variant	1%	2%	5%	10%	Attacks caught
Full EBR	87.7	90.5	91.7	91.1	5 / 7 / 8 / 8
No diminishing returns	51.5	67.3	77.8	77.8	2 / 5 / 6 / 8
Tiers and knapsack only	51.1	63.4	75.5	76.5	1 / 4 / 6 / 8
No rarity score	83.8	88.6	91.1	92.3	5 / 7 / 8 / 8
No automation score	87.1	89.9	90.5	91.7	5 / 7 / 8 / 8
No pivoting	87.7	90.5	91.7	91.1	5 / 7 / 8 / 8
No turn heads	87.7	90.5	91.7	91.1	5 / 7 / 8 / 8
No ledger	87.7	90.5	91.7	92.3	7 / 7 / 8 / 8

HONEST NOTE

Diminishing returns carries most of the benefit. Pivoting and turn heads made no measurable difference to signature recall on these captures, and rarity and automation each changed it only slightly when removed alone. They may overlap, so that when one is removed another covers for it. This study does not show that those parts are needed, and a simpler EBR might do just as well. The ledger costs detection at 1%, which is the price of exact flow counts.

28 Everything we got wrong

This chapter is here on purpose. A result is only believable if its failures are visible.

Table 28.1. The mistakes, and what corrected them.

What we thought	What turned out to be true
A weighted evidence score must beat a simple method	An unweighted comparator beat it on several captures, which led to the EBR redesign
Later-published rules would favour EBR	They did not. New rules mostly fire on flows old rules had already flagged
SMB was the main cause of the red-team failure	Scan flows taking 82% of the budget were the main cause. SMB was a smaller second cause
Gaps inside flows caused lost detections	Disproved by an isolation experiment. The login flows alert at every level when scanned alone
A steeper repetition discount was the fix	It damaged other captures when tested on all 13 earlier cases
The scan cap would help on scan-heavy data	It failed its pre-registered test. It works as a safeguard
The SMB change had a small cost	That difference was within scan noise, so the claim was withdrawn
Signature recall never changes between scans	It moved by 0.4 points on average, and 12.5 on a small case
The blind-spot estimate was the project's most novel finding	It did worse than guessing on the sealed data. It remains an open idea
EBR alone always leads an IDS	It led in two tests and fell behind at 1% and 2% on scan-heavy traffic

BIG IDEA

Most of these were found by testing an explanation instead of trusting it. The most useful habit in the whole project was asking "how would I know if this were wrong?"

28.1 A favourite surprise

The "FTP brute force" attack in the CIC dataset turned out to be 192,300 refused connections, because the server was not listening. Keeping a small set of exemplars and an exact count was the right forensic answer, and the case exposed a flaw in our first evaluation metric.

Part VI

What it means, and how to use it

Whether the idea is new, where it falls short, how it could be used for real, and how to run and defend the project yourself.

Novelty · Limits · Real-world use · Running the project · Viva questions

29 What is new, and what is not

"Novel" is a word that gets people into trouble. This chapter sets out exactly what can be claimed.

29.1 Related work

Table 29.1. The closest prior work, and how EBR differs.

Prior work	What it does	How EBR differs
Time Machine [5, 6]	A per-connection cutoff, with storage classes an IDS can adjust	Value-driven levels per flow, no IDS needed, an exact byte cap, and turn heads
XNET [7]	Dynamic sampling at very high speed, favouring rare destinations and DNS	Learns from the capture alone, reads content, and optimises against a byte budget. XNET runs live, while EBR runs offline
Conditional logging [11], IDS-triggered PCAP [8]	Keep traffic that matches rules	No rules at selection time, so later detections stay possible
TrimPCAP [9], selective capping [10]	Fixed trimming per session or packet	Trim depth varies per flow according to value
Patents on value-based retention	Value-based retention as a general concept	Retaining by value is not new. These were seen through search summaries, not read in full

29.2 The claim you can defend

> **BIG IDEA**
>
> In a bounded literature search, we found no published method that combines signature-free surprisal with Good–Turing correction, two-level diminishing returns, per-flow fidelity selection as a knapsack, and a budget-charged ledger, or that verifies a reduction against an IDS and repairs it inside the same budget before deletion.

The word that matters is *combines*. The individual ideas have relatives, and the contribution lies in how they are put together and how they were tested. Two contributions stand out:

1. **Behaviour-aware diminishing returns**, which the ablation study shows makes retention work under a tight cap.
2. **Verify before delete inside a fixed budget**, which gave the largest sealed-tested gain.

29.3 Claims you should not make

BE CAREFUL

- **"This has never been done."** A bounded search cannot show that.
- **"Value-based retention is a new idea."** The patents above contradict this.
- **"EBR beats an IDS."** Two small tests and mixed results on alert instances cannot show this in general.
- **"Repair works on unknown attacks."** It only protects what a rule can already see.

CHAPTER IN SHORT

The combination is new as far as a bounded search can tell. Verify before delete is the strongest novel angle, and it is the one with sealed evidence behind it.

30 The limits of EBR

Every method has limits. Knowing them is a sign that you understand the work.

Table 30.1. Known limitations.

Limit	What it means
Not an intrusion detector	The suspect list helps triage. It is not proof of compromise
Discarded data is gone	Bulk file contents and most encrypted payload cannot be recovered from the reduced copy
Works offline	It makes two passes over a finished capture. A live version is future work
Reads classic PCAP	PCAPNG files must be converted first
Login sprays look like repeats	Diminishing returns cannot see that identical-looking logins carry different passwords
Rate-based rules are hard to repair	They need many flows close together, which can cost more than the budget allows
Low-and-slow attacks are harder	A handful of quiet flows produce little surprise to work with
The scan cap is unproven	It failed its test. Treat it as a safeguard
Repair needs an IDS and time	The original must be scanned before deletion
The tests are small	A few independent cases, so intervals are wide. Many come from one generated lab dataset
Settings are design choices	Tier weights and powers were tuned on development data and could suit other networks less well
A rule match is not attribution	A “Ponmocup” alert means a rule matched, not that Ponmocup was present
Detection is not investigation	We measured what an IDS can find. We did not measure what a human analyst can reconstruct
Reduced files can raise new alerts	Missing packets can change what an IDS sees. Every method showed a few of these
Hashes are not chain of custody	They show files have not changed, not that evidence would be admissible

31 Could this work in the real world?

EBR is a research prototype. This chapter sketches a real deployment, and that design is untested.

31.1 A three-tier storage design



Figure 31.1. A proposed way to use EBR. The storage budget buys longer memory, not less evidence.

For the same disk space, a 5% cap means keeping twenty days of reduced evidence for each day of full recording. EBR is a way of buying time.

31.2 Can it keep up?

WORKED EXAMPLE · Rough arithmetic from our measurements

EBR took about 20 seconds per GB to profile and score, and Suricata about 62 seconds per GB to scan. That is roughly 12 MB per second, so one machine like ours could keep up with about 100 megabits per second of sustained traffic. A full 1 gigabit link would need about ten times that. These figures come from different runs, so this is an estimate.

31.3 What would come next

1. **See credentials.** Make content comparison notice login attempts with different passwords, so sprays stop looking like repeats. This is the most valuable next step.
2. **Retest the blind-spot estimate** on captures with more later-published rules.
3. **Try other networks**, especially a real organisation's traffic, and build a streaming version.
4. **Measure real analysts.** Give investigators reduced evidence and time how well they do.
5. **Simplify.** Test whether a smaller EBR, without the parts that showed no benefit, does as well.

CHAPTER IN SHORT

EBR suits a nightly batch that turns a short full-packet buffer into months of reduced evidence. Large links would need batches running across several machines, and the real-world design is untested.

32 Running the project yourself

Everything needed to show the project working, and where to find things.

32.1 What is in the folder

IS project/

- `START_HERE.md` the first file to open
- `Run_FORENSIC_DEMO.cmd` double-click to run the demonstration
- `EBR_ALGORITHM.md` technical summary and viva answers
- `report/` this guide and the IEEE paper, with their sources
- `final/` all the working code, data and results
  - `ebr/` the algorithm: profile, value, allocate, retain, verify, baselines, forensic, report
  - `lab/` Suricata lab setup and checks
  - `tests/test_ebr.py` 45 automated tests
  - `data/forensic/` the captures
  - `results/forensic/` every test's saved results and reports
  - `results/forensic_demo/` two finished demonstration runs
  - `forensic_demo.py` the demonstration
  - `forensic_test.py` the main test harness
  - `sealed3_plan.json`, `FREEZE*.json` the pre-registered plan and the seals

Figure 32.1. The project folder.

32.2 The demonstration

The demonstration shrinks real captures with EBR, then has Suricata investigate what is left.

```
python -B final/forensic_demo.py --quick       one 1.5 GB case, about a minute
python -B final/forensic_demo.py               11.3 GB with labelled attacks, 5% cap
python -B final/forensic_demo.py --sealed      7 GB never seen during tuning
python -B final/forensic_demo.py --cap 0.02    a harder cap
```

Or double-click `Run_FORENSIC_DEMO.cmd`. The report opens in your browser when it finishes.

Table 32.1. Demonstration results already saved in the project, at a 5% cap.

Run	Reduced	Signatures kept	Labelled attacks
Default, labelled captures	11.28 GB to 564.2 MB (95.0% smaller)	143 of 147	8 of 8
`--sealed`, unseen captures	7.03 GB to 296.0 MB (95.8% smaller)	82 of 87	1 of 1

> **BE CAREFUL · What the demonstration does and does not show**
>
> The demonstration runs **EBR alone**, without the repair loop, so do not present its numbers as the 99.6% result. The default captures also helped shape the current version, so that run is a demonstration and not a clean test. Three labelled attacks in it raise no alert even on the complete capture.

32.3 Checking the work

```
python -B -m unittest final/tests/test_ebr.py    the 45 automated tests
python -B final/freeze.py check v3               recompute the seal of the third test
```

32.4 Setting up on a new PC

The demonstration needs the Suricata lab. On a Windows PC with WSL 2, create it once, then fetch the captures:

```
powershell -ExecutionPolicy Bypass -File final\lab\setup_lab.ps1
python -B final/fetch_forensic_data.py
```

32.5 Where to find each result

Table 32.2. Saved reports.

Result	File
Third sealed test	`final/results/forensic/sealed3/report.html`
Second sealed test	`final/results/forensic/sealed2/report.html`
First sealed test	`final/results/forensic/test/report.html`

33 Viva questions

The questions you are most likely to be asked, with short answers you can say out loud. Read them aloud once before the viva.

33.1 The basics

Q. What problem does this project solve?

Full packet captures are the best evidence after an attack, but they are too large to keep for long. We reduce them to a fixed cap of 1 to 10% while keeping the evidence an investigator needs.

Q. What is EBR, in one sentence?

It is an algorithm that decides how much of each network conversation to keep, based on how much evidence those bytes add, measured in bits of surprise and discounted for repetition.

Q. Why not just use gzip?

Gzip keeps everything, but it still needed about 40–56% of the original size on our test sets. EBR reaches 1–10% by choosing what to give up, and the two can be combined.

Q. What does "evidence-based" mean here?

Each byte is kept according to how much evidence it adds. It does not mean that EBR detects attacks.

Q. Isn't this just anomaly detection?

It uses anomaly signals, but its output is a retention decision. A wrong guess costs a few kilobytes of storage, not an analyst's time, so imperfect scores are affordable.

33.2 The algorithm

Q. Why measure value in bits?

Rarity, automation and pivoting then share one unit, so they can be combined without arbitrary weights between them.

Q. Why does Good–Turing matter?

Without it, a spam bot's thousands of unique DNS names all look very rare and take the whole budget. It accounts for the fact that if most values are unique, a new unique value is expected.

Q. Which part matters most?

Diminishing returns. Removing it cut signature recall at a 1% cap from 88% to 52%. Pivoting and turn heads showed no measurable benefit here, so a simpler version may do just as well.

Q. Why give the first 16 flows full value?

Rate-based IDS rules need several events close together. Evenly spread single samples would not trigger them.

Q. Is the knapsack solution optimal?

The greedy method is optimal for the fractional relaxation, and within one flow's final segment of the true integer optimum. With the scan cap added it becomes a good heuristic. The value model is itself a design choice, so "optimal" only means optimal for that model.

Q. Why charge the ledger to the budget?

Otherwise EBR would get free storage that the comparators do not, and the comparison would be unfair.

33.3 The tests

Q. How do you know it works?

An independent IDS with 52,778 public rules judged what survived, on 24.5 GB of real captures across three sealed tests. EBR never saw those rules.

Q. What does sealed mean?

We hashed the code, settings and scoring before the test data was opened, so nothing could be tuned to the answers. Anyone can recompute those hashes.

Q. Why pre-register?

We wrote the hypotheses and the pass rule down, hashed them, and only then opened the data. That stops results being explained away afterwards.

Q. Did you tune it on the test data?

No. Tuning used development cases only. Each test used captures opened after the freeze was recorded.

Q. Is EBR better than everything else?

Not on its own. It led an IDS in the first two tests, but on scan-dominated captures it fell well behind at 1% and 2%. With the repair loop, it beat the IDS method at 2%, 5% and 10% in a pre-registered sealed test, and the 1% result was inconclusive. There are few cases, so this is strong evidence and not proof.

Q. Did your fix work?

Not as hoped. The scan cap fixed the capture that exposed the flaw, but it failed its pre-registered test on fresh scan-heavy captures. It helped 3 of 8 cases and hurt none on signature recall, so we keep it as a safeguard.

Q. How many hypotheses passed?

One of five in full. Three failed and one could not be scored. We report that as it came out, because a result is only credible if its failures are visible.

Q. What was your most surprising finding?

The "FTP brute force" in the dataset is 192,300 refused connections, because the server was not listening. Keeping a few exemplars and an exact count was the right forensic answer.

33.4 Verify before delete

Q. What is verify before delete?

The original still exists until it is deleted. So we scan both copies, restore what was lost within the same budget, and only then delete. At a 5% cap it kept 99.6% of attack signatures, against 84.7% for EBR alone.

Q. Doesn't that need the IDS you were trying to avoid?

The first selection uses no rules, so unknown threats still depend on EBR. The IDS only repairs what it can see, and it runs once per batch instead of steering capture live.

Q. Does the blind-spot estimate work?

It looked promising on unsealed data, then did worse than guessing on the few usable pairs in the sealed test. It is an open idea, not a finding.

33.5 Limits and novelty

Q. What is the biggest weakness?

Login sprays. Diminishing returns treats attempts that differ only in the password as repeats, so EBR alone loses them. On one capture it lost 110 of 158 attack alert instances at a 10% cap.

Q. What exactly is novel?

The combination, and verifying inside a fixed budget before deletion. We found no published method that does this in a bounded search. Retaining by value is not new, and we do not claim it is.

Q. Would you use this in production?

Not yet. It is an offline prototype tested on few cases. It would need testing on real networks and engineering for speed.

Q. What would you do next?

Make content comparison see credentials, so sprays stop looking like repeats. Then try other networks, build a streaming version, and measure real analysts.

A Glossary

Every technical term in the book, in plain words.

Term	Meaning
ACK	A TCP flag meaning "I received that".
Ablation	Removing one part of a system to see what that part was contributing.
Alert	What an IDS produces when a rule matches.
Alert instance	In this project, one distinct combination of rule and pair of hosts.
Attack signature	In this project, one distinct rule from an attack category that fired.
Automation (score)	EBR's measure of how machine-made a group of flows looks.
Baseline	What is normal for this capture. Also used for a simple method you compare against.
Bit	The smallest unit of data. In information theory, the information in one fair coin flip.
Bit-reversed order	A visiting order that keeps any prefix of a list spread evenly across it.
Bootstrap	Estimating uncertainty by resampling your own data many times.
Botnet	A group of infected computers controlled by one attacker.
Brute force	Guessing passwords over and over.
Budget, cap	The maximum size a reduced capture may have, such as 5% of the original.
Bulk	Everything in a flow past its head, deep head and turn heads.
C2	Command and control: how an attacker gives orders to infected machines.
Chain of custody	A documented record of who handled evidence and when.
Client	The computer that asks.
Client fingerprint	A summary of the software a client seems to be using.
Comparator	A method tested against yours.
Compression	Shrinking a file. Lossless compression can rebuild the original exactly.
Content skeleton	A flow's request with words removed and punctuation kept.
Cross-site scripting	An attack that plants code in a page for other visitors to run.
DDoS	A denial-of-service attack from many machines at once.
Deep head	The first 16 KB of each direction of a flow (Tier 2).
Development data	Data used to tune a method. Never used for final results.
Diminishing returns	The idea that each extra copy of something adds less value.
DNS	The internet's phone book. It turns names into IP addresses.
DoS	Denial of service: overwhelming a service so real users cannot reach it.
EBR	Evidence-Based Retention, the algorithm in this project.
Elephant flow	A flow holding a huge number of bytes.
Encryption	Scrambling data so only the intended reader can understand it.
Ephemeral port	The temporary high-numbered port a client uses.
ET Open	Emerging Threats Open, a free public set of detection rules.
Exemplar	One of the first flows of a behaviour, kept at full value.
Exploit	An attack that takes advantage of a software bug.
False positive	An alert on harmless traffic.
Fidelity tier	A level of detail at which part of a flow can be kept.
Five-tuple	The five values that identify a flow: two addresses, two ports, one protocol.
FIN	A TCP flag meaning "I am finished".
Flow	All the packets of one conversation.
Flow cutoff	A method that keeps only the first part of each flow.
Flow ledger	EBR's small compressed table recording every flow.

Term	Meaning
Forensics	Careful examination of data to work out what happened.
Freeze	Recording the hashes of files so that any later change can be detected.
Good–Turing	A way of estimating how likely rare or unseen things are.
Gzip	A common lossless compression tool.
Hash	A short fingerprint of a file.
Head	The first 1 KB of each direction of a flow (Tier 1).
Header	Control information at the front of a packet.
HTTP, HTTPS	The protocol of the web. HTTPS is the encrypted version.
Hull (upper concave)	The levels of a flow left after removing poor-value ones.
Hypothesis	A precise prediction that a test can pass or fail.
IDS	Intrusion detection system: software that raises alerts on suspicious traffic.
IDS-triggered capture	Keeping only traffic that raised an alert.
IP address	The number that identifies a device on a network.
Knapsack problem	Choosing items of various values and sizes to fit a limit.
Lateral movement	An attacker moving between computers inside a network.
Ledger	See flow ledger.
Login spray	Trying common passwords against many devices or accounts.
Lossy reduction	Shrinking data by deliberately dropping some of it.
Metadata	Data about data, such as who talked to whom and when.
Mouse flow	A very small flow.
Novelty (here)	The share of values in a capture that were seen exactly once.
Packet	A small piece of data sent across a network.
Payload	The content carried by a packet.
PCAP	The standard file format for recorded packets.
Pivot	Moving from one suspicious item to everything connected with it.
Port	A number that picks out one program on a computer.
Port scan	Trying many ports or addresses to see what answers.
Pre-registration	Writing down predictions and pass rules before seeing the data.
Protocol	An agreed set of rules for communicating.
Rate-based rule	An IDS rule that counts events over time.
Recall	The share of something that survived.
Red team	People who attack a network on purpose, to practise or test defences.
Repair	Restoring the flows behind lost detections, inside the same budget.
Router	A device that passes messages towards their destination.
Rule	A description of bad traffic used by an IDS.
Sealed test	A test whose code and settings were locked before its data was opened.
Sequence number	TCP's numbering of bytes, used to reorder them and detect gaps.
Server	The computer that answers.
SHA-256	The hash function used in this project.
Skeleton	Control packets that prove a connection happened (Tier 0).
SMB	Windows file-sharing protocol.
SQL injection	An attack that gets text treated as a database command.
SSH	Encrypted remote control of a computer.
Streaming	Processing data as it is read, without holding it all in memory.
Surprisal	The information in an event, $-\log_2 P$, measured in bits.
Suricata	The open-source IDS used as the judge in this project.
SYN	A TCP flag meaning "can we talk?".
TCP	The reliable transport protocol, which uses a handshake.
Telnet	Old, unencrypted remote control of a device.

Term	Meaning
Time travel (here)	Testing old captures with rules published later.
TLS	The encryption protocol behind HTTPS.
Turn	One side's stretch of talking in a conversation.
Turn heads	The first 1 KB of each later turn in a flow (Tier 3).
UDP	A simple transport protocol with no handshake or resending.
Verify before delete	Checking and repairing a reduction before deleting the original.
Wireshark	A free program for looking inside PCAP files.

B Formula sheet

Every formula in EBR on one page.

Table B.1. The formulas.

Name	Formula
Budget	$B = \lfloor \text{cap} \times \text{original bytes} \rfloor$
Probability	$P(x) = \max\left(n_x/N,\ \text{novelty}/n_x\right)$, where novelty is the share of values seen once
Surprisal	$s(x) = \min\left(16,\ -\log_2 P(x)\right)$
Excess	$e_t = \max\left(0,\ s_t - \text{median}(s_t)\right)$ for each trait t
Rarity	$R = e_{(1)} + 0.25 \sum_{i>1} e_{(i)}$, where $e_{(1)}$ is the largest excess
Automation	$A = \log_2(n) \times \max(1 - \text{spread},\ \text{regularity},\ \text{failure share})$ for groups of $n \geq 8$ flows
Host mass	$m_h = \sum 2^{\min(b-6,\ 12)}$ over flows h started with $b = \max(R, A) \geq 6$ bits
Host suspicion	$H_h = \max\left(0,\ \log_2(1 + m_h) - \text{median host score}\right)$
Pivot	$U = 0.5 \times \max(H_{\text{client}},\ H_{\text{server}})$. For a victim's next rare peers within 30 minutes, at least $0.25 \times H_{\text{attacker}}$
Discount	r^{-p}, with $r = 1$ for the first 16 flows and then $r = \text{rank} - 15$
Powers	$p = 0.5$ for repeats and silent flows, $p = 0.25$ for variations
Multiplier	$M = M_{\text{repeat}} \times \min(M_{\text{client}},\ M_{\text{server}},\ M_{\text{silent}})$
Flow value	$V = \left(1 + \max(R, A) + U\right) \times M$
Level value	Value of a level $= V \times$ sum of the weights of occupied tiers up to that level
Order of selection	Hull segments are sorted by value per byte, highest first
Repair boost	A flow behind a lost alert gets $V = 1000 \times \max(V)$

Settings

The values in `final/ebr_config.json`, which are sealed by `FREEZE_v3.json`.

Table C.1. EBR settings.

Setting	Value	Meaning
`tcp_idle_seconds`	300	Quiet time that ends a TCP flow
`udp_idle_seconds`	60	Quiet time that ends a UDP flow
`head_bytes`	1,024	Size of the head, each way
`deep_head_bytes`	16,384	Size of the deep head, each way
`turn_head_bytes`	1,024	Bytes kept from each later turn
`udp_tier_packets`	8, 64	UDP head and deep head, in packets
`tier_weights`	0.2, 0.5, 0.15, 0.1, 0.05	Share of value for each tier
`opaque_bulk_factor`	0.1	Multiplier past the head for unreadable flows
`readable_apps`	smb	Unencrypted protocols treated as readable
`silent_budget_share`	0.3	First-pass limit for payload-free flows
`rarity_cap_bits`	16	Most surprise one trait can give
`rarity_tail_weight`	0.25	Weight of the less surprising traits
`automation_min_flows`	8	Smallest group that can count as automated
`pivot_seed_bits`	6	Flows at or above this feed host suspicion
`pivot_weight`	0.5	Lift from a suspicious host
`pivot_victim_weight`	0.25	Lift for a victim's next moves
`pivot_window_seconds`	1,800	How long the victim lift lasts
`pivot_rare_peer_flows`	4	A peer seen this often or less is rare
`exemplar_burst`	16	Flows of a behaviour that keep full value
`repeat_power`	0.5	Steep discount
`spread_power`	0.25	Gentle discount
`base_value_bits`	1.0	Starting value of every flow
`ledger_max_share`	0.1	Largest share of the budget for the ledger

The repair loop's settings are a boost of 1,000, up to 64 neighbouring flows, and two repair rounds.

References

[1] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.

[2] I. J. Good, "The population frequencies of species and the estimation of population parameters," *Biometrika*, vol. 40, nos. 3–4, pp. 237–264, 1953.

[3] P. Sinha and A. A. Zoltners, "The multiple-choice knapsack problem," *Oper. Res.*, vol. 27, no. 3, pp. 503–515, 1979.

[4] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman & Hall, 1993.

[5] S. Kornexl, V. Paxson, H. Dreger, A. Feldmann, and R. Sommer, "Building a time machine for efficient recording and retrieval of high-volume network traffic," in *Proc. ACM SIGCOMM Internet Meas. Conf.*, 2005, pp. 267–272.

[6] G. Maier, R. Sommer, H. Dreger, A. Feldmann, V. Paxson, and F. Schneider, "Enriching network security analysis with time travel," in *Proc. ACM SIGCOMM*, 2008, pp. 183–194.

[7] T. Papastergiou *et al.*, "XNET: Intelligent dynamic sampling for high-speed network security monitoring," arXiv:2608.18349, Aug. 2026.

[8] Z. Gaster, "Rethinking full packet capture: Evaluating IDS-triggered PCAP," SANS Institute, Sep. 2026.

[9] E. Hjelmvik, "Don't delete PCAP files – trim them!" Netresec, Dec. 2017.

[10] V. Uceda, M. Rodríguez, J. Ramos, J. L. García-Dorado, and J. Aracil, "Selective capping of packet payloads for network analysis and management," in *Proc. Traffic Monit. Anal. (TMA)*, LNCS 9053, 2015.

[11] Open Information Security Foundation, "Suricata user guide." [Online]. Available: https://docs.suricata.io

[12] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, 2010, pp. 305–316.

[13] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 1093–1110.

[14] K. Kent, S. Chevalier, T. Grance, and H. Dang, "Guide to integrating forensic techniques into incident response," NIST, Gaithersburg, MD, USA, SP 800-86, Aug. 2006.

[15] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Nat. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, 2018.

[16] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.

[17] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops*, 2021, pp. 7–12.

[18] S. García, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *Comput. Secur.*, vol. 45, pp. 100–123, 2014.

[19] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Zenodo, 2020. [Online]. Available: https://zenodo.org/record/4743745